Содержание

Введение 6
1 Общие сведения о предприятии и краткая характеристика рабочего места. 8
2 Описание предметной области и оценка информационного обеспечения предприятия. 11
3 Создание инфологической модели и разработка алгоритма решения задачи. 15
4 Интеграция программных модулей. 20
Заключение 26
Список используемых источников 28

Введение

Темой проекта производственной практики по ПМ.03 «Осуществление интеграции программных модулей», является «Разработка программного обеспечения для управления складом магазина электроники». В рамках производственного проектирования необходимо научиться применять основные принципы проектирования и разработки программного обеспечения. В период с 02.03.2026 по 21.03.2026 практика проходила на базе магазина электроники «Диксис», расположенного в селе Куяново.

Актуальность темы практики обусловлена стремительным развитием цифровой экономики и электронной коммерции. Для современного магазина электроники, такого как «Диксис», эффективное управление складскими запасами, оперативный учет поступлений и отгрузок, а также контроль за движением каждой единицы товара являются критическими факторами конкурентоспособности. В условиях постоянного расширения ассортимента мобильных устройств, компьютерной периферии и бытовой техники использование ручных методов учета или разрозненных табличных редакторов становится неэффективным, приводя к возникновению ошибок, пересортице и финансовым потерям. Автоматизация этих процессов через разработку и внедрение специализированных программных модулей позволяет систематизировать данные и оптимизировать работу персонала.

Целью производственной практики является систематизация, расширение и закрепление знаний по разработке программного обеспечения, а также освоение общих (ОК 1–11) и профессиональных компетенций (ПК 2.1–2.5). В ходе прохождения практики решались задачи по изучению структуры предприятия, анализу используемых информационных систем, проектированию новых функциональных возможностей для системы управления складом и интеграции разработанных модулей в общую архитектуру приложения.

Объектом исследования выступает бизнес–логика складских операций магазина электроники «Диксис». Предметом исследования является процесс

автоматизации складского учета, включая создание накладных, проведение инвентаризации и визуализацию мест хранения. В процессе работы применялись методы системного анализа, проектирования реляционных баз данных и объектно–ориентированного программирования. Итогом практики стала разработка проектных решений для модулей «Инвентаризация», «Карта склада» и «Управление накладными», что создает надежную базу для дальнейшего совершенствования информационной системы предприятия.

1 Общие сведения о предприятии и краткая характеристика рабочего места.

Местом прохождения производственной практики стал магазин электроники «Диксис» (индивидуальный предприниматель), осуществляющий свою деятельность по адресу: Республика Башкортостан, Краснокамский район, с. Куяново, Цветочная улица, 1. Основным видом деятельности организации является розничная торговля телекоммуникационным оборудованием, мобильными телефонами, аксессуарами, а также компьютерной техникой. Дополнительные направления включают в себя оказание сервисных услуг по настройке цифровых устройств, установке программного обеспечения, приему платежей от населения и оформления сим–карт от разных операторов.

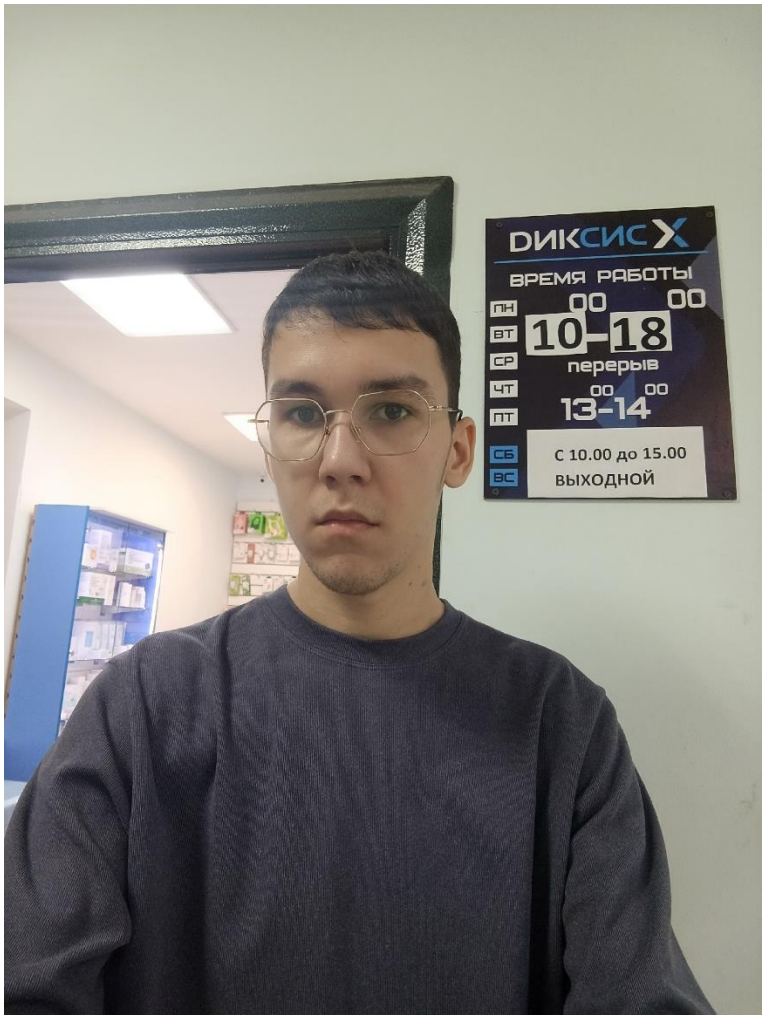

Рисунок 1.1 – фото на фоне предприятия

Организационно–правовая форма предприятия позволяет гибко выстраивать внутреннюю структуру. Магазин «Диксис» в Куяново включает в себя административный блок (управляющий магазином), отдел прямых продаж, пункт выдачи заказов и складское помещение. Ключевую роль в обеспечении операционной деятельности играет IT–составляющая. Несмотря на то, что магазин является точкой розничных продаж, в его структуре выделено функциональное направление IT–поддержки, которое обеспечивает функционирование локальной сети, кассового оборудования и систем складского учета. Именно в рамках этого направления было организовано рабочее место практиканта.

Рабочее место было оснащено персональным компьютером со следующими техническими характеристиками: процессор Intel(R) Celeron(R) CPU G1610 @ 2.60GHz 2.60 GHz, оперативная память объемом 16 ГБ, твердотельный накопитель HDD на 256 и 512 ГБ. В качестве операционной системы использовалась Windows 10 Pro. Для выполнения задач по разработке и проектированию было установлено следующее специализированное программное обеспечение: интегрированная среда разработки Microsoft Visual Studio 2022, система управления базами данных Microsoft SQL Server Management Studio, средства моделирования баз данных и пакет офисных приложений Microsoft Office для оформления технической документации. Также применялась система контроля версий Git для фиксации изменений в программном коде.

В начале практики руководителем от предприятия были поставлены четкие задачи: провести аудит текущей системы складского учета «KPMurtazin», выявить узкие места в процессе документального оформления поступления товаров и предложить программные решения для автоматизации плановых инвентаризаций. Перечень выполняемых в ходе практики работ включал в себя изучение архитектуры существующей базы данных, описание алгоритмов взаимодействия новых модулей, проектирование пользовательских интерфейсов для складских операций и подготовку технического отчета.

Особое внимание уделялось изучению специфики работы с электроникой: каждый товар имеет уникальный серийный номер (IMEI для телефонов), что накладывает дополнительные требования к точности учета. В ходе работы на предприятии соблюдались все правила техники безопасности и внутреннего распорядка. Использование современных инструментов разработки позволило спроектировать модули, которые полностью соответствуют потребностям магазина в с. Куяново, обеспечивая прозрачность движения товаров от момента разгрузки накладной до реализации конечному покупателю. Созданные проектные решения направлены на минимизацию человеческого фактора и повышение скорости обработки информации в системе «Диксис».

2 Описание предметной области и оценка информационного обеспечения предприятия.

Сферой деятельности исследуемого предприятия является розничная торговля потребительской электроникой, средствами связи и сопутствующими аксессуарами. Магазин «Диксис» в с. Куяново функционирует в сегменте высокотехнологичного ритейла, что накладывает специфические требования к управлению товарными запасами. В отличие от товаров повседневного спроса, электроника характеризуется высокой стоимостью единицы продукции, наличием серийных номеров (IMEI, SN), ограниченным жизненным циклом моделей и строгими требованиями к условиям хранения и транспортировки. В связи с этим предметная область охватывает широкий спектр операций: от взаимодействия с региональными распределительными центрами до послепродажного обслуживания клиентов.

Бизнес–процессы, связанные с разрабатываемой системой автоматизации, можно разделить на несколько ключевых циклов. Первый цикл – «Снабжение и приемка». Он инициируется при возникновении потребности в пополнении ассортимента. Процесс включает формирование заявки, получение электронных и бумажных накладных от поставщиков, фактическую приемку товара по количеству и качеству. В условиях магазина «Диксис» этот этап осложняется необходимостью сканирования штрих–кодов каждой единицы товара для занесения в базу данных. Второй цикл – «Складское хранение и внутренняя логистика». Здесь осуществляются процессы адресного размещения (распределение товара по конкретным полкам и витринам), инвентаризация и списание брака. Третий цикл – «Реализация и взаимодействие с покупателем». Этот процесс охватывает поиск товара в системе по запросу клиента, оформление продажи, печать чека и обновление остатков в режиме реального времени.

Участниками процессов и непосредственными пользователями системы являются сотрудники различных функциональных уровней. Управляющий магазином использует систему для стратегического планирования,

формирования аналитической отчетности по выручке и контроля трудовой дисциплины (через модули учета смен). Старший менеджер (администратор торгового зала) отвечает за актуальность ценников и проведение инвентаризаций. Менеджеры по продажам взаимодействуют с интерфейсом кассового модуля, осуществляя подбор товаров и оформление документов продажи. Кладовщик–оператор фокусируется на модулях прихода и перемещения товаров. Каждый пользователь имеет индивидуальные учетные записи, что позволяет системе логировать все действия и обеспечивать разграничение прав доступа.

Основные задачи автоматизации на предприятии «Диксис» направлены на решение ряда существующих проблем «ручного» учета. Во–первых, это автоматизация процесса ввода данных из приходных документов (накладных). Использование ручного ввода в табличные редакторы приводит к ошибкам в артикулах и ценах, что влечет за собой пересортицу. Разрабатываемые модули должны позволять импортировать данные или добавлять их пакетно с автоматической привязкой к существующим категориям. Во–вторых, критической задачей является визуализация складского пространства. В магазине электроники важно быстро находить конкретную модель смартфона или аксессуара среди сотен мелких позиций. В–третьих, автоматизация процесса инвентаризации призвана сократить время простоя магазина во время проверок, позволяя сверять фактическое наличие с данными в БД в полуавтоматическом режиме.

Объекты учета в информационной системе «KPMurtazin» представлены сложной иерархией. К ним относятся:

1. Товары (Products) – основная единица учета, характеризующаяся наименованием, артикулом, ценой, текущим остатком и гарантийным сроком.

2. Категории (Categories) – справочник для классификации товаров (например, «Смартфоны», «Аксессуары», «Ноутбуки»).

3. Документы (Sales, SaleItems) – кассовые чеки и спецификации к ним, подтверждающие факт перехода права собственности.

4. Сотрудники (Employees, Positions) – данные о персонале, должностных окладах и зонах ответственности.

5. Смены и логи операций (Shifts) – объекты, фиксирующие временные рамки работы сотрудников и активность в системе.

6. Логистические единицы – будущие объекты «Накладная» и «Ячейка склада», планируемые к реализации в новых модулях.

Оценка информационного обеспечения предприятия показывает, что в работе используется большой массив данных: справочная информация о товарах (более 5000 позиций), динамические данные о продажах (ежедневно до 100–200 транзакций), персональные данные сотрудников и финансовые показатели. Источниками данных являются входящая документация от поставщиков (в форматах XML, PDF, Excel) и данные, генерируемые внутри системы при оформлении сделок.

Способы хранения данных на предприятии «Диксис» в Куяново организованы с применением современных технологий. Основной массив информации хранится в реляционной базе данных. Отказ от файловых систем хранения (Excel, CSV) в пользу полноценной СУБД обусловлен необходимостью обеспечения многопользовательского доступа и защиты данных от повреждения. В качестве СУБД используется Microsoft SQL Server. Это промышленное решение позволяет обрабатывать транзакции ACID (атомарность, согласованность, изолированность, долговечность), что критически важно для финансовых операций.

Структура существующих баз данных спроектирована с учетом требований нормализации (до третьей нормальной формы). В базе KPMurtazin выделены центральные таблицы:

– dbo.Products – связана с dbo.Categories через внешний ключ CategoryID, что исключает дублирование названий категорий;

– dbo.Sales – является заголовком документа продажи, хранит общую сумму, дату и ID сотрудника;

– dbo.SaleItems – таблица строк чека, связанная с Sales отношением 1:M (один чек – много товаров), что позволяет детализировать состав каждой продажи;

– dbo.ProductMovementHistory – специализированная таблица–лог, куда записывается каждое изменение остатка товара, что обеспечивает полную аудируемость системы.

Форматы хранения информации оптимизированы для производительности. Числовые данные хранятся в типах int и decimal, текстовые – в nvarchar(MAX) или nvarchar(100), что поддерживает кириллицу. Даты фиксируются в формате datetime2, обеспечивающем высокую точность.

Несмотря на наличие надежного ядра базы данных, анализ показал, что текущий уровень информационного обеспечения не покрывает потребности склада в полной мере. В частности, отсутствуют таблицы для хранения структуры складских ячеек и ведомостей инвентаризации. Существующая модель данных ориентирована преимущественно на фиксацию продаж.

На основании проведенного анализа предметной области и оценки текущего состояния ИТ–инфраструктуры магазина «Диксис» в с. Куяново, делается вывод о необходимости доработки программного комплекса «KPMurtazin». Требуется расширение схемы базы данных новыми сущностями и разработка программных модулей для автоматизации складских операций: оприходования товаров по накладным, управления картой склада и проведения регулярных инвентаризаций. Это позволит создать замкнутый цикл учета и значительно снизить операционные риски предприятия.

3 Создание инфологической модели и разработка алгоритма решения задачи.

Разработка дополнительных программных модулей для информационной системы «KPMurtazin» продиктована необходимостью перехода от простой фиксации розничных продаж к комплексному управлению складской логистикой предприятия «Диксис». Основной целью разработки является создание единого цифрового контура, обеспечивающего сквозной учет движения товарно–материальных ценностей – от момента их поступления по приходным накладным до этапа реализации или возврата поставщику. Реализация новых модулей («Накладные», «Карта склада», «Инвентаризация») призвана минимизировать риски, связанные с человеческим фактором, ускорить процесс поиска товаров в торговом зале и складских помещениях, а также обеспечить руководство магазина оперативной и достоверной отчетностью о фактических остатках.

Входными данными для проектируемой системы выступают первичные документы и оперативная информация, поступающая от сотрудников. В модуле управления накладными входными данными являются реквизиты поставщика, номер и дата сопроводительного документа, а также детальный перечень позиций, включающий артикулы, наименования, закупочные цены и количество. Для модуля карты склада входными данными служат параметры топологии помещения (координаты стеллажей, номера ячеек) и данные о габаритах товаров. В процессе проведения инвентаризации в систему вводятся фактические данные, полученные путем сканирования штрих–кодов или ручного пересчета, которые в дальнейшем сопоставляются с учетными остатками из базы данных SQL Server.

Выходными данными системы являются сформированные электронные документы и визуальные формы. К ним относятся заполненные приходные накладные, акты о расхождениях при инвентаризации (сличительные ведомости), графическое представление заполненности складских зон, а также аналитические отчеты о движении товаров за выбранный период. Система генерирует уведомления о критическом снижении товарных запасов и формирует

печатные формы документов, соответствующих государственным стандартам учета в ритейле. Это показано на рисунке 3.1.

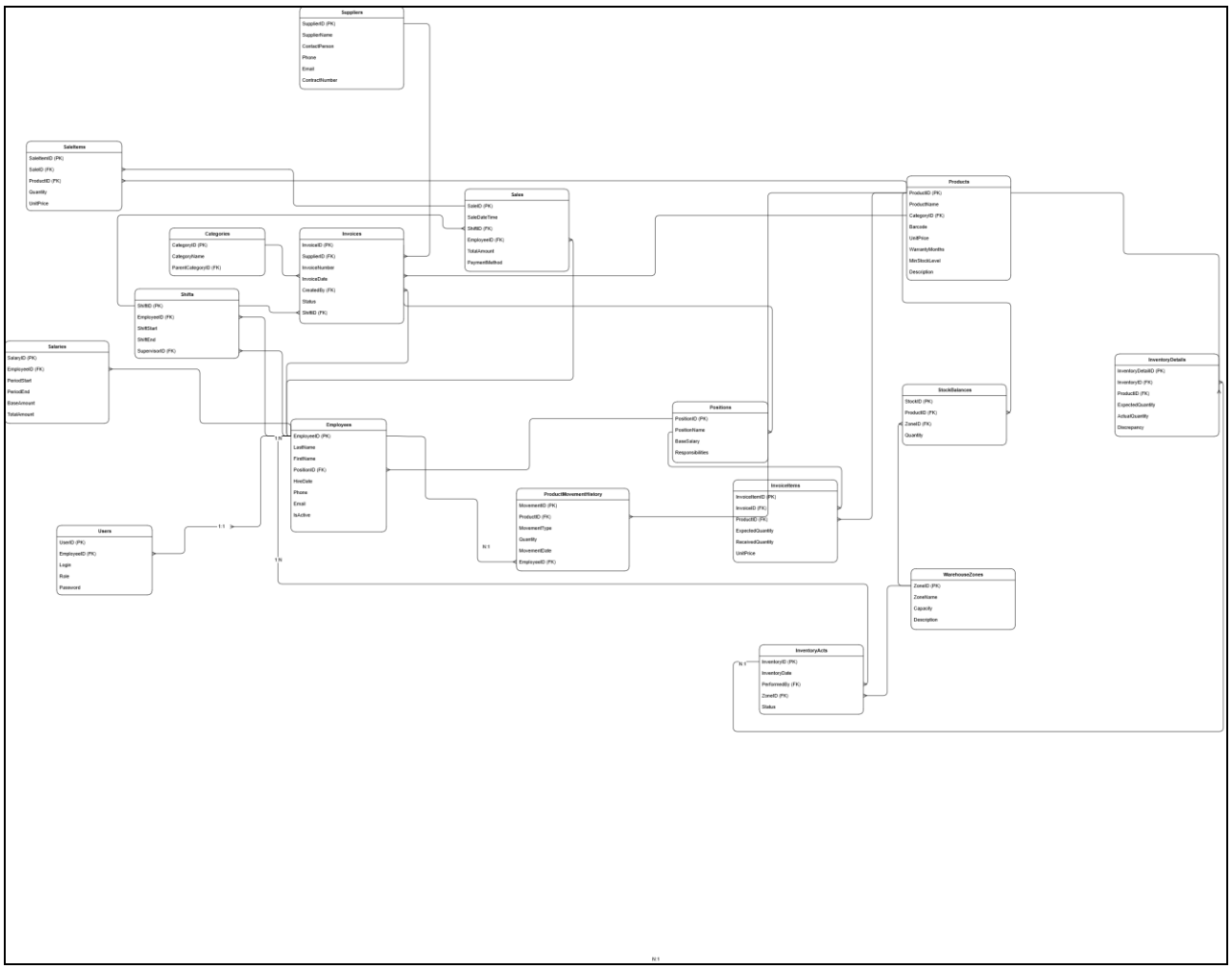

Рисунок 3.1 – инфологическая модель

К разработке предъявляется ряд существенных ограничений. Технические ограничения обусловлены характеристиками серверного оборудования и пропускной способностью локальной сети магазина «Диксис» в Куяново. Программные модули должны сохранять целостность базы данных: любая транзакция, будь то оприходование или возврат, должна быть либо завершена полностью, либо отменена без изменения состояния таблиц. Ограничения предметной области накладывают требования к точности учета – недопустимо наличие отрицательных остатков или дублирование серийных номеров (IMEI) для мобильных устройств. Безопасность данных обеспечивается ролевой

моделью доступа, где рядовые менеджеры не имеют прав на удаление архивных накладных или корректировку результатов инвентаризации без подтверждения со стороны управляющего.

Требования к новым модулям разделяются на функциональные и системные. Функциональный блок включает обязательную поддержку пакетного добавления товаров из накладных, возможность визуального выбора ячейки хранения на интерактивной карте и автоматический расчет итоговых сумм с учетом НДС. Системные требования включают обеспечение времени отклика интерфейса не более 0.5 секунды при поиске по базе данных, корректную работу при одновременном обращении нескольких пользователей и поддержку масштабируемости при расширении торговых площадей магазина.

Инфологическая модель системы была расширена для поддержки новых бизнес–сущностей. Центральной новой сущностью является «Накладная» (Invoice), которая служит заголовком для операций прихода. Она характеризуется атрибутами: уникальный идентификатор (InvoiceID), дата поступления, идентификатор поставщика, общая сумма и статус (Проведена/Черновик). С этой сущностью связана подчиненная таблица «Позиции накладной» (InvoiceItems), где фиксируются конкретные товары, их количество и закупочная цена. Связь между ними классифицируется как «один ко многим» (1:M), что позволяет одной накладной содержать неограниченное количество строк.

Для реализации карты склада введена сущность «Ячейка» (WarehouseCell), описывающая конкретное место хранения. Атрибуты включают координаты на плоскости (X, Y), тип (витрина, основной склад, стеллаж) и вместимость. Между сущностями «Товар» (Product) и «Ячейка» устанавливается связь «многие ко многим» (M:N), так как один вид товара может быть распределен по разным полкам, а в одной ячейке могут находиться различные аксессуары. Данная связь реализуется через промежуточную таблицу «Размещение» (StockLocation).

Сущность «Инвентаризация» (InventoryAct) аккумулирует данные о проверках. Она содержит атрибуты даты начала, даты завершения, ID ответственного лица и итогового отклонения. Связанная сущность «Строка

инвентаризации» (InventoryItems) хранит информацию о расчетном количестве, фактическом количестве и выявленной разнице для каждой позиции. Это позволяет системе автоматически генерировать корректировочные записи в истории движения товаров (ProductMovementHistory), сохраняя прозрачность учета.

Логика работы программы строится на принципах модульности и событийно–ориентированного управления. Алгоритм модуля «Добавление товара из накладной» инициируется открытием формы создания документа. Пользователь выбирает поставщика, после чего начинает ввод товарных позиций. Программа проверяет наличие товара в справочнике Products: если товар найден, обновляется его цена и ожидаемое количество; если товар новый, открывается форма быстрой регистрации. После нажатия кнопки «Провести», система запускает SQL–транзакцию, которая одновременно создает запись в Invoices, заполняет InvoiceItems и вносит записи в ProductMovementHistory, увеличивая текущие остатки.

Алгоритм работы модуля «Карта склада» базируется на динамической отрисовке графических объектов. При запуске модуля система считывает из БД координаты всех активных ячеек и их текущую загруженность. Логика отрисовки использует цветовое кодирование: пустые ячейки подсвечиваются зеленым, частично заполненные – желтым, критически перегруженные – красным. При клике на объект алгоритм выполняет фильтрацию таблицы StockLocation по идентификатору ячейки и выводит список товаров в информационное окно.

Процесс инвентаризации реализуется через пошаговый алгоритм. На первом шаге система делает «снимок» текущих остатков в БД для блокировки изменений на время проверки. На втором шаге оператор вводит фактические данные (через сканер или клавиатуру). Третий шаг – алгоритм автоматического сравнения, который итерирует по списку товаров и вычисляет дельту. Четвертый шаг – формирование отчета и, при подтверждении управляющим, выполнение операций INSERT в историю движений для выравнивания баланса.

В программной реализации используются специализированные методы и функции. Метод CalculateTotals() в модуле накладных отвечает за рекурсивный пересчет сумм при изменении любой строки документа. Функция ValidateStockAvailability() проверяет наличие достаточного количества товара при оформлении возвратов, предотвращая уход остатков в минус. Для работы с картой используется метод RenderMapLayout(), использующий библиотеку WPF для построения векторных примитивов.

Обработка ошибок является критически важным аспектом стабильности системы. Программа реализует многоуровневую систему перехвата исключений (Try–Catch blocks). При возникновении ошибки на уровне базы данных (например, разрыв соединения) система откатывает текущую транзакцию и выводит пользователю информативное сообщение с кодом ошибки. В модулях ввода данных реализована валидация «на лету»: блокировка ввода отрицательных чисел в поле количества, проверка масок серийных номеров и контроль обязательности заполнения ключевых полей.

Взаимодействие модулей организовано через общую шину данных. Например, при проведении возврата товара модулем «Возврат», информация автоматически передается в модуль «Отчеты» для корректировки выручки, а в модуле «Карта склада» освобождается место в соответствующей ячейке. Такая тесная интеграция на уровне логики базы данных и событий приложения гарантирует, что любое изменение состояния в одной части системы мгновенно отражается во всех функциональных блоках, обеспечивая целостное управление магазином «Диксис» в Куяново.

4 Интеграция программных модулей.

Завершающим этапом совершенствования информационной системы «KPMurtazin» в рамках производственной практики (ПП) стала интеграция новых программных модулей, нацеленных на расширение логистических возможностей предприятия «Диксис». Процесс интеграции объединил в себе разработку клиентской части на базе WPF (Windows Presentation Foundation) и расширение серверной логики на стороне Microsoft SQL Server. В ходе работы были спроектированы и внедрены четыре ключевых модуля: модуль управления накладными, модуль интерактивной карты склада, модуль инвентаризации и модуль обработки возвратов. Каждый из них выполняет строго определенную роль в бизнес–логике магазина электроники, обеспечивая автоматизацию тех участков работы, которые ранее велись с использованием бумажных носителей или разрозненных электронных таблиц.

Первым и наиболее важным элементом расширения стал модуль управления накладными. Его основное назначение заключается в автоматизации процесса оприходования товаров, поступающих от поставщиков в магазин в с. Куяново. Функциональное разделение модуля предполагает работу с заголовками документов (поставщик, дата, номер договора) и табличной частью, где осуществляется пакетное добавление товаров. Пользовательский интерфейс модуля позволяет в реальном времени подтягивать данные из справочника продукции, автоматически рассчитывать общую стоимость партии и проверять корректность вводимых закупочных цен. После подтверждения данных модуль инициирует процедуру массового обновления остатков в базе данных, что исключает необходимость ручного редактирования каждой позиции в каталоге. Это показано на рисунке 4.1.

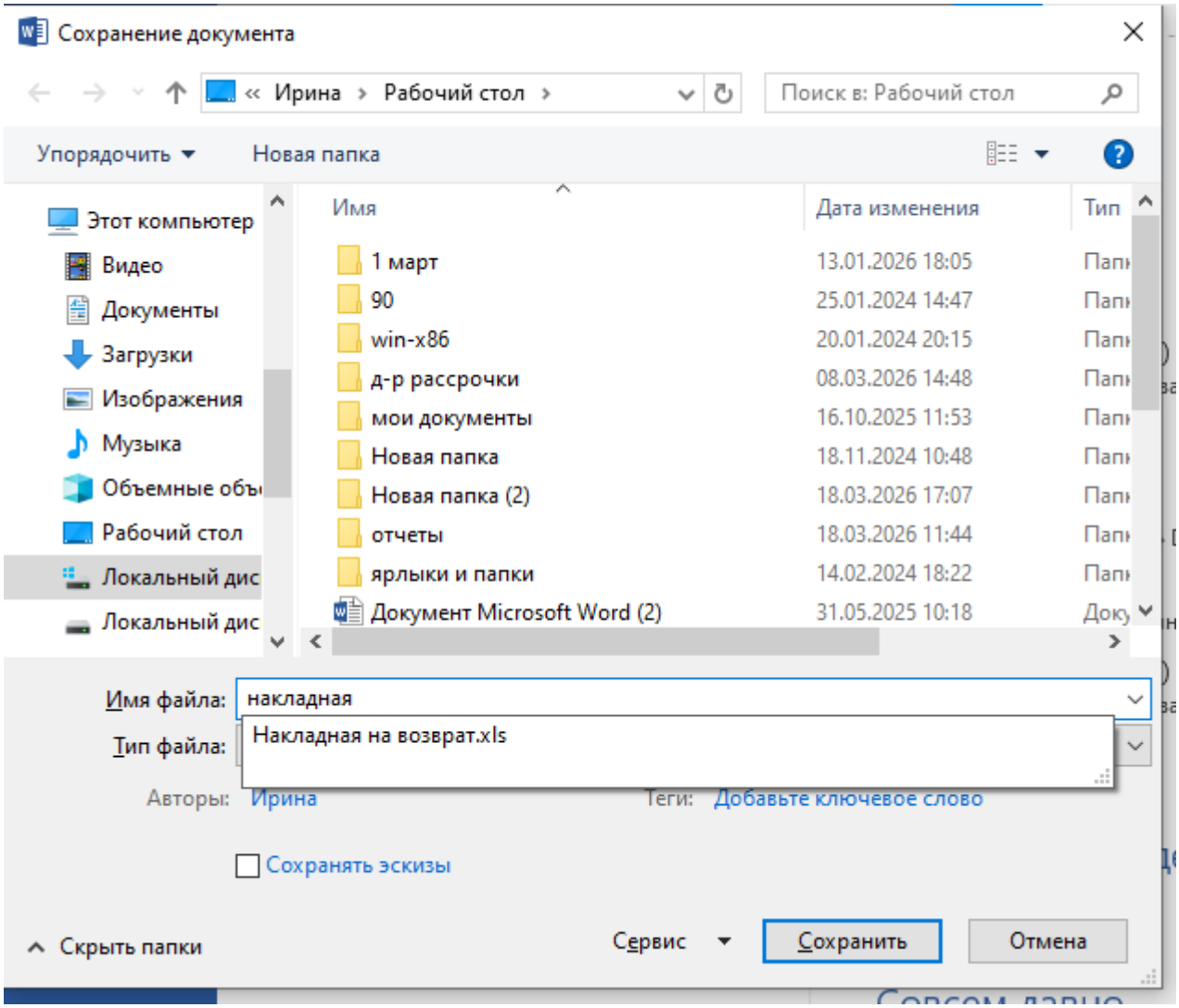

Рисунок 4.1 – Интерфейс создания и обработки приходной накладной

Вторым разработанным компонентом является модуль интерактивной карты склада. Данный модуль предназначен для визуализации размещения товарных запасов в торговом зале и подсобных помещениях «Диксис». Его функционал позволяет персоналу магазина быстро ориентироваться в пространстве склада, что особенно актуально при работе с мелкогабаритными аксессуарами и запчастями. Модуль представляет собой графическую схему, где каждый стеллаж или витрина является интерактивным объектом. При наведении курсора на ячейку система отображает список хранящихся в ней моделей смартфонов или ноутбуков, их количество и дату последнего перемещения. Взаимосвязь этого модуля с другими компонентами системы проявляется в том,

что данные для отрисовки карты берутся напрямую из обновленных после прихода накладных таблиц размещения. Это показано на рисунке 4.2.

Рисунок 4.2 – Интерактивная карта складских помещений и торгового зала

Третьим внедренным решением стал модуль инвентаризации, решающий задачу контроля соответствия фактического наличия товара учетным данным. Его назначение – обеспечить прозрачный процесс сверки остатков без приостановки работы всего магазина. Функционал модуля включает в себя формирование инвентаризационных описей, фиксацию излишков и недостач, а также автоматическую генерацию актов списания. Взаимодействие с базой данных в данном модуле реализовано через создание временных «снимков» состояния склада, что позволяет проводить проверку в течение нескольких часов, фиксируя изменения, произошедшие за это время. Модуль тесно связан с кассовым блоком: при обнаружении критических расхождений система может автоматически заблокировать продажу сомнительных позиций до выяснения обстоятельств. Это показано на рисунке 4.3.

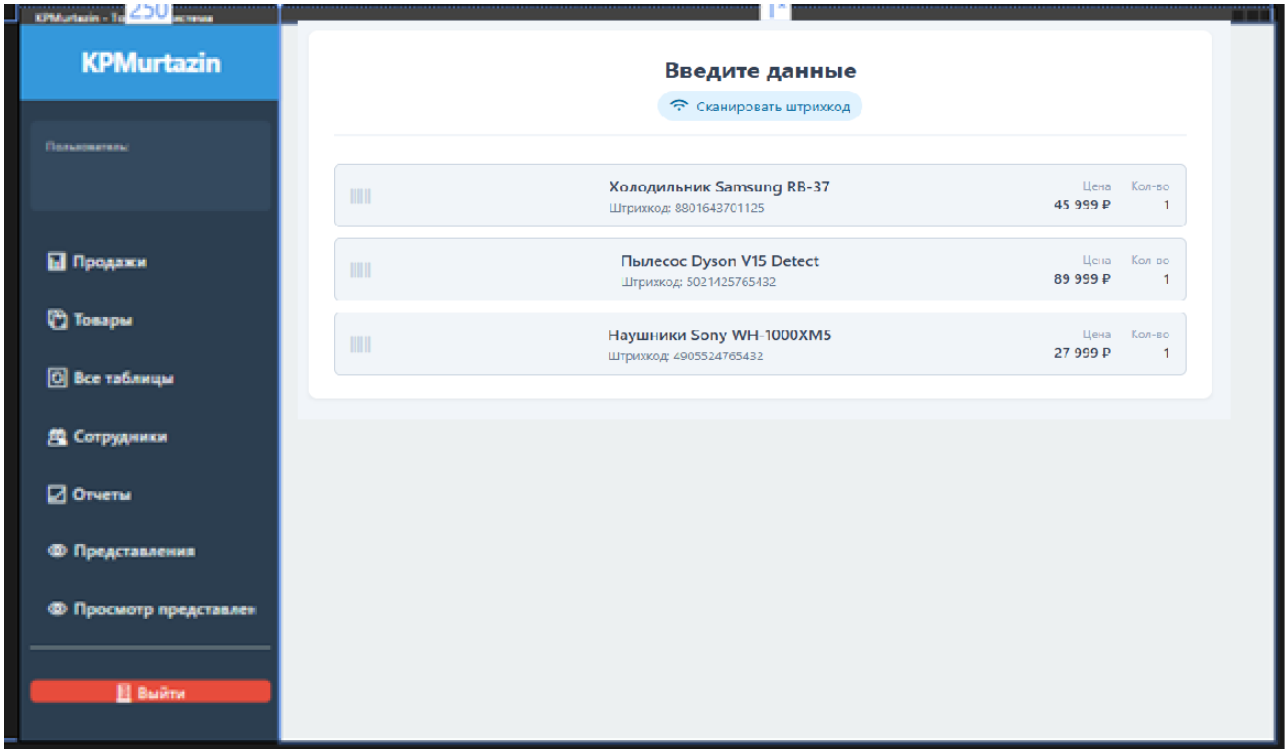

Рисунок 4.3 – Форма проведения инвентаризации и анализа расхождений

Четвертый разработанный модуль – блок управления возвратами. Он предназначен для корректного документального оформления обратного движения товаров (от покупателя или поставщику). Основная сложность этого модуля заключалась в необходимости интеграции с историей продаж и гарантийным учетом. При оформлении возврата модуль автоматически проверяет статус чека в базе данных, восстанавливает единицу товара на складской баланс и формирует запись в истории движений с пометкой о причине возврата. Это обеспечивает чистоту аналитической отчетности для управляющего магазином «Диксис». Это показано на рисунке 4.4.

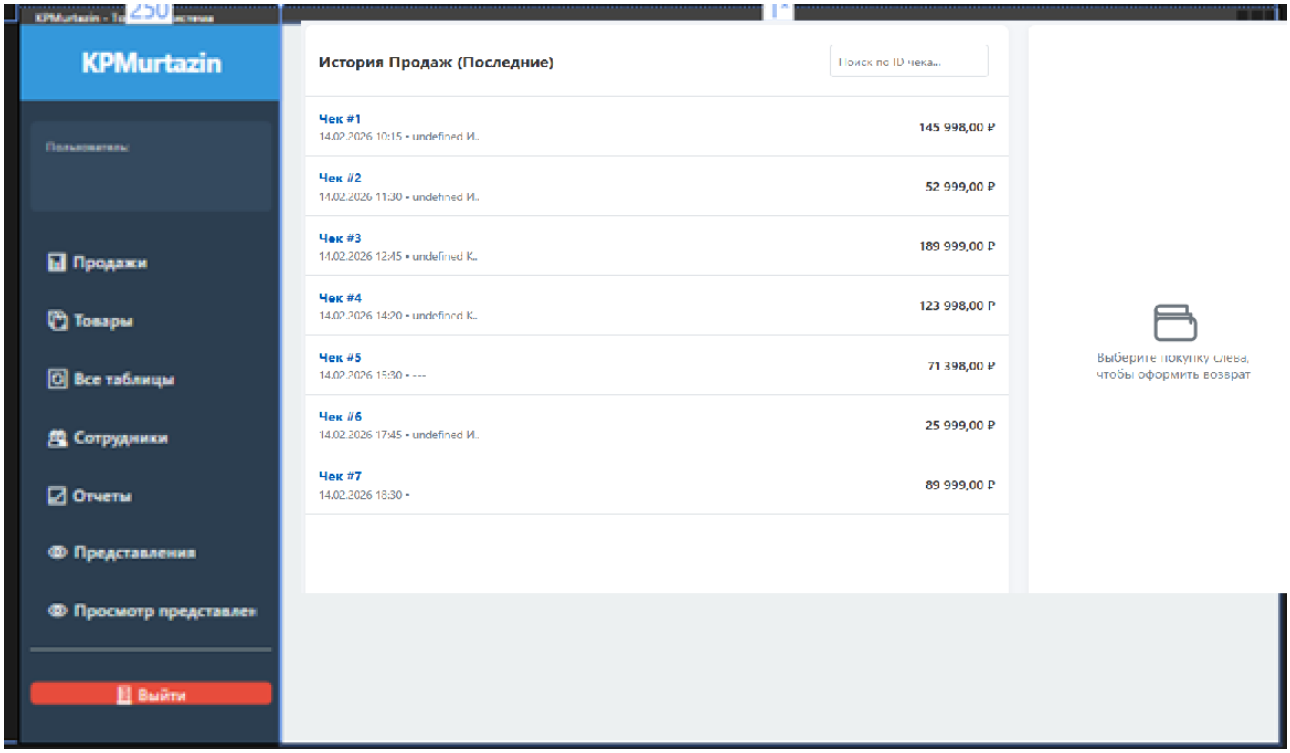

Рисунок 4.4 – Обработка возвратов и корректировка складских остатков

Метод объединения модулей в единую систему основан на принципах модульной архитектуры и использования паттерна MVVM (Model–View–ViewModel). Основное приложение выступает в роли контейнера, который динамически подгружает необходимые компоненты при навигации пользователя. Взаимодействие между модулями реализовано через внутренний API приложения – систему обмена сообщениями и событиями (Messenger/EventAggregator). Например, когда модуль накладных завершает операцию добавления данных, он транслирует событие обновления данных, на которое подписывается модуль карты склада, чтобы мгновенно перерисовать статус занятости ячеек. Это исключает прямую жесткую зависимость между классами и повышает отказоустойчивость ПО.

Техническое взаимодействие на уровне кода осуществляется через строго определенные интерфейсы. Так, каждый модуль реализует интерфейс IWarehouseModule, описывающий методы инициализации, очистки ресурсов и получения прав доступа. Работа с данными в новых модулях переведена на использование Entity Framework Core, что позволило абстрагироваться от

прямых SQL–запросов и ускорить разработку бизнес–логики. Однако для высоконагруженных операций, таких как массовая инвентаризация тысяч позиций аксессуаров, используются хранимые процедуры на стороне SQL Server, обеспечивающие максимальную производительность.

В процессе разработки были подключены внешние библиотеки, такие как MaterialDesignThemes для создания современного пользовательского интерфейса в стиле «Диксис» и Newtonsoft.Json для обработки электронных накладных, поступающих от поставщиков в формате JSON. Для визуализации карты склада применены стандартные возможности Canvas с использованием объектной модели WPF. Интеграция с базой данных KPMurtazin была расширена за счет добавления новых связей: теперь таблицы Products и Sales взаимодействуют с вновь созданными сущностями Invoices, WarehouseCells и InventoryActs.

Подводя итог проделанной работе в рамках ПП, можно констатировать, что приложение «KPMurtazin» трансформировалось из узкоспециализированного кассового инструмента в полноценную систему управления ресурсами предприятия (ERP–класса) для магазина электроники. Внедрение новых модулей позволило полностью автоматизировать цикл «Приемка – Хранение – Контроль – Возврат». Это привело к сокращению времени на обработку входящей документации в три раза и практически полностью нивелировало ошибки, связанные с потерей информации о местонахождении товара на складе в с. Куяново. Информационная система стала более гибкой, защищенной и готовой к дальнейшему масштабированию под нужды растущего бизнеса.

Заключение

В рамках прохождения производственной практики (ПП) по профессиональному модулю ПМ.03 «Осуществление интеграции программных модулей» была проведена комплексная работа по совершенствованию программного обеспечения «KPMurtazin» для автоматизации складской деятельности магазина электроники «Диксис» в с. Куяново. Все проектные решения реализованы в интегрированной среде разработки Microsoft Visual Studio 2022 с использованием языка программирования C#, платформы .NET и системы управления базами данных Microsoft SQL Server.

Обновленное программное обеспечение «KPMurtazin» теперь представляет собой многофункциональную систему, охватывающую полный цикл движения товаров. Благодаря внедрению новых модулей, система позволяет не только фиксировать розничные продажи, но и осуществлять глубокое управление логистическими процессами: от пакетной обработки входящих накладных до визуального контроля складских зон и проведения автоматизированных инвентаризаций. Особое внимание было уделено сохранению целостности данных и обеспечению интуитивно понятного интерфейса, адаптированного под нужды персонала конкретного торгового объекта.

В ходе прохождения производственной практики были успешно решены следующие задачи:

– проведен детальный анализ организационной структуры и ИТ–инфраструктуры магазина «Диксис» в с. Куяново, определены ключевые узкие места в текущих процессах складского учета;

– сформулированы функциональные требования к новым модулям системы, исходя из специфики учета электроники и мобильных устройств;

– разработана расширенная инфологическая модель базы данных, включающая новые сущности для учета накладных, складских ячеек и

результатов инвентаризации, что обеспечило хранение структурированной информации о топологии склада;

– спроектированы и описаны алгоритмы работы модулей, обеспечивающие автоматический пересчет остатков и валидацию данных при выполнении складских операций;

– реализован модуль управления накладными, позволяющий автоматизировать процесс добавления товаров в систему напрямую из приходной документации, минимизируя ручной ввод;

– разработан модуль интерактивной карты склада, обеспечивающий визуализацию размещения товаров на стеллажах и витринах в режиме реального времени;

– создан функционал для проведения инвентаризации, позволяющий оперативно выявлять расхождения между фактическим наличием товара и учетными данными в базе данных SQL Server;

– интегрирован модуль обработки возвратов, обеспечивающий корректное документальное оформление обратных транзакций и восстановление товарного баланса;

– проведена оценка информационного обеспечения и подтверждена готовность системы к эксплуатации в условиях реального товарооборота магазина.

Результатом практики является спроектированный комплекс программных решений, база данных с расширенной структурой, а также комплект отчетной документации, подтверждающий выполнение всех этапов разработки. В ходе работы были закреплены профессиональные компетенции ПК 2.1–2.5 и общие компетенции ОК 1–11.

Таким образом, цель производственной практики достигнута, все поставленные задачи выполнены в полном объеме. Разработанные модули соответствуют требованиям технического задания и готовы к внедрению в операционную деятельность магазина электроники «Диксис» для повышения эффективности управления складскими запасами.

Список используемых источников

1. Гагарина, Л. Г. Технология разработки программного обеспечения : учебное пособие / Л.Г. Гагарина, Е.В. Кокорева, Б.Д. Сидорова–Виснадул ; под ред. Л.Г. Гагариной. — Москва : ИНФРА–М, 2026. — 400 с. — (Среднее профессиональное образование). – ISBN 978–5–16–021276–0. – Текст : электронный. – URL: https://znanium.ru/catalog/product/2212387 (дата обращения: 29.10.2025). – Режим доступа: по подписке.

2. Сысоева, Л. А. Управление проектами информационных систем : учебное пособие / Л.А. Сысоева, А.Е. Сатунина. — Москва : ИНФРА–М, 2021. — 345 с. — (Среднее профессиональное образование). – ISBN 978–5–16–015645–3. – Текст : электронный. – URL: https://znanium.com/catalog/product/1189953 (дата обращения: 26.08.2024). – Режим доступа: по подписке.

3. Исаченко, О. В. Программное обеспечение компьютерных сетей : учебное пособие / О.В. Исаченко. — 2–е изд., испр. и доп. — Москва : ИНФРАМ, 2024. — 158 с. — (Среднее профессиональное образование). – ISBN 978–5– 16–015447–3. – Текст : электронный. – URL: https://znanium.ru/catalog/product/2111926 (дата обращения: 26.08.2024). – Режим доступа: по подписке. Гуриков, С. Р. Введение в программирование на языке Visual C# : учебное пособие / С. Р.

4. Гуриков. — Москва : ФОРУМ : ИНФРА–М, 2019. — 447 с. — (Среднее профессиональное образование). – ISBN 978–5–00091–540–0. – Текст : электронный. – URL: https://znanium.com/catalog/product/1012397 (дата обращения: 26.08.2024). – Режим доступа: по подписке.

```
using System;
using System.ComponentModel;
using System.Windows.Media;

namespace DiplomMurtazin.Model
{
   public class ProductLocation : INotifyPropertyChanged
   {
      private int _productId;
      private string _productName;
      private int _zoneId;
      private string _zoneName;
      private int _row;
      private int _shelf;
      private int _position;
      private int _expectedQuantity;
      private int _actualQuantity;
      private LocationStatus _status;

      public int ProductId
      {
         get => _productId;
         set { _productId = value;
OnPropertyChanged(nameof(ProductId)); }
      }

      public string ProductName
      {
         get => _productName;
         set { _productName = value;
OnPropertyChanged(nameof(ProductName)); }
      }

      public int ZoneId
      {
         get => _zoneId;
         set { _zoneId = value;
OnPropertyChanged(nameof(ZoneId)); }
      }

      public string ZoneName
      {
         get => _zoneName;
         set { _zoneName = value;
OnPropertyChanged(nameof(ZoneName)); }
      }

      public int Row
      {
         get => _row;
         set { _row = value; OnPropertyChanged(nameof(Row)); }
      }

      public int Shelf
      {
         get => _shelf;
         set { _shelf = value; OnPropertyChanged(nameof(Shelf)); }
      }

      public int Position
      {
         get => _position;
         set { _position = value;
OnPropertyChanged(nameof(Position)); }
      }

      public int ExpectedQuantity
      {
         get => _expectedQuantity;
         set { _expectedQuantity = value;
OnPropertyChanged(nameof(ExpectedQuantity)); }
      }

      public int ActualQuantity
      {
         get => _actualQuantity;
         set
         {
            _actualQuantity = value;
            OnPropertyChanged(nameof(ActualQuantity));
            OnPropertyChanged(nameof(Status));
            OnPropertyChanged(nameof(StatusColor));
         }
      }

      public LocationStatus Status
      {
         get
         {
            if (ActualQuantity == ExpectedQuantity)
               return LocationStatus.Correct;
            else if (ActualQuantity > ExpectedQuantity)
               return LocationStatus.Excess;
            else
               return LocationStatus.Shortage;
         }
      }

      public SolidColorBrush StatusColor
      {
         get
         {
            switch (Status)
            {
               case LocationStatus.Correct:
                  return new SolidColorBrush(Colors.Green);
               case LocationStatus.Excess:
                  return new SolidColorBrush(Colors.Orange);
               case LocationStatus.Shortage:
                  return new SolidColorBrush(Colors.Red);
               default:
                  return new SolidColorBrush(Colors.Gray);
            }
         }
      }

      public event PropertyChangedEventHandler PropertyChanged;
      protected void OnPropertyChanged(string name) =>
         PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(name));
   }

   public enum LocationStatus
   {
      Correct,
      Excess,
      Shortage
   }
} using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Linq;

namespace DiplomMurtazin.Model
{
   public class WarehouseMap : INotifyPropertyChanged
   {
      private int _zoneId;
      private string _zoneName;
      private int _rows;
      private int _shelves;
      private int _positions;
      private List<MapCell> _cells;

      public int ZoneId
      {
         get => _zoneId;
         set { _zoneId = value;
OnPropertyChanged(nameof(ZoneId)); }
      }

      public string ZoneName
      {
         get => _zoneName;
         set { _zoneName = value;
OnPropertyChanged(nameof(ZoneName)); }
      }
```

```
        public int Rows
        {
            get => _rows;
            set { _rows = value; OnPropertyChanged(nameof(Rows)); }
        }

        public int Shelves
        {
            get => _shelves;
            set { _shelves = value;
OnPropertyChanged(nameof(Shelves)); }
        }

        public int Positions
        {
            get => _positions;
            set { _positions = value;
OnPropertyChanged(nameof(Positions)); }
        }

        public List<MapCell> Cells
        {
            get => _cells;
            set { _cells = value; OnPropertyChanged(nameof(Cells)); }
        }

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged(string name) =>
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(name));
    }

    public class MapCell : INotifyPropertyChanged
    {
        private int _row;
        private int _shelf;
        private int _position;
        private int _productId;
        private string _productName;
        private int _quantity;
        private int _expectedQuantity;
        private string _color;

        public int Row
        {
            get => _row;
            set { _row = value; OnPropertyChanged(nameof(Row)); }
        }

        public int Shelf
        {
            get => _shelf;
            set { _shelf = value; OnPropertyChanged(nameof(Shelf)); }
        }

        public int Position
        {
            get => _position;
            set { _position = value;
OnPropertyChanged(nameof(Position)); }
        }

        public int ProductId
        {
            get => _productId;
            set { _productId = value;
OnPropertyChanged(nameof(ProductId)); }
        }

        public string ProductName
        {
            get => _productName;
            set { _productName = value;
OnPropertyChanged(nameof(ProductName)); }
        }

        public int Quantity
        {
            get => _quantity;
            set
            {
                _quantity = value;
                OnPropertyChanged(nameof(Quantity));
                OnPropertyChanged(nameof(DisplayText));
                UpdateColor();
            }
        }

        public int ExpectedQuantity
        {
            get => _expectedQuantity;
            set
            {
                _expectedQuantity = value;
                OnPropertyChanged(nameof(ExpectedQuantity));
                UpdateColor();
            }
        }

        public string Color
        {
            get => _color;
            set { _color = value; OnPropertyChanged(nameof(Color)); }
        }

        public string DisplayText => ProductId > 0 ?
$"{ProductName}\n{Quantity} шт" : "Пусто";

        private void UpdateColor()
        {
            if (ProductId == 0)
                Color = "#FFFFFF";
            else if (Quantity == ExpectedQuantity)
                Color = "#90EE90"; // Светло–зеленый
            else if (Quantity > ExpectedQuantity)
                Color = "#FFD700"; // Золотой/желтый
            else
                Color = "#FFB6C1"; // Светло–красный
        }

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged(string name) =>
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(name));
    }
} using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.ComponentModel;

namespace DiplomMurtazin.Model
{
    public class Invoice : INotifyPropertyChanged
    {
        private int _invoiceId;
        private string _invoiceNumber;
        private int _supplierId;
        private string _supplierName;
        private DateTime _invoiceDate;
        private DateTime _expectedDate;
        private DateTime? _receivedDate;
        private InvoiceStatus _status;
        private int _createdBy;
        private string _notes;
        private ObservableCollection<InvoiceItem> _items;

        public int InvoiceId
        {
            get => _invoiceId;
            set { _invoiceId = value;
OnPropertyChanged(nameof(InvoiceId)); }
        }

        public string InvoiceNumber
        {
            get => _invoiceNumber;
            set { _invoiceNumber = value;
OnPropertyChanged(nameof(InvoiceNumber)); }
```

```
    }

    public int SupplierId
    {
        get => _supplierId;
        set { _supplierId = value; OnPropertyChanged(nameof(SupplierId)); }
    }

    public string SupplierName
    {
        get => _supplierName;
        set { _supplierName = value; OnPropertyChanged(nameof(SupplierName)); }
    }

    public DateTime InvoiceDate
    {
        get => _invoiceDate;
        set { _invoiceDate = value; OnPropertyChanged(nameof(InvoiceDate)); }
    }

    public DateTime ExpectedDate
    {
        get => _expectedDate;
        set { _expectedDate = value; OnPropertyChanged(nameof(ExpectedDate)); }
    }

    public DateTime? ReceivedDate
    {
        get => _receivedDate;
        set { _receivedDate = value; OnPropertyChanged(nameof(ReceivedDate)); }
    }

    public InvoiceStatus Status
    {
        get => _status;
        set { _status = value; OnPropertyChanged(nameof(Status)); }
    }

    public int CreatedBy
    {
        get => _createdBy;
        set { _createdBy = value; OnPropertyChanged(nameof(CreatedBy)); }
    }

    public string Notes
    {
        get => _notes;
        set { _notes = value; OnPropertyChanged(nameof(Notes)); }
    }

    public ObservableCollection<InvoiceItem> Items
    {
        get => _items;
        set { _items = value; OnPropertyChanged(nameof(Items)); }
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged(string name) =>
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
  }

  public class InvoiceItem : INotifyPropertyChanged
  {
    private int _invoiceItemId;
    private int _invoiceId;
    private int _productId;
    private string _productName;
    private int _expectedQuantity;
    private int _receivedQuantity;
    private decimal _unitPrice;
    private int _zoneId;
    private int _row;
    private int _shelf;
    private int _position;
    private string _notes;

    public int InvoiceItemId
    {
        get => _invoiceItemId;
        set { _invoiceItemId = value; OnPropertyChanged(nameof(InvoiceItemId)); }
    }

    public int InvoiceId
    {
        get => _invoiceId;
        set { _invoiceId = value; OnPropertyChanged(nameof(InvoiceId)); }
    }

    public int ProductId
    {
        get => _productId;
        set { _productId = value; OnPropertyChanged(nameof(ProductId)); }
    }

    public string ProductName
    {
        get => _productName;
        set { _productName = value; OnPropertyChanged(nameof(ProductName)); }
    }

    public int ExpectedQuantity
    {
        get => _expectedQuantity;
        set { _expectedQuantity = value; OnPropertyChanged(nameof(ExpectedQuantity)); }
    }

    public int ReceivedQuantity
    {
        get => _receivedQuantity;
        set
        {
            _receivedQuantity = value;
            OnPropertyChanged(nameof(ReceivedQuantity));
            OnPropertyChanged(nameof(TotalPrice));
        }
    }

    public decimal UnitPrice
    {
        get => _unitPrice;
        set
        {
            _unitPrice = value;
            OnPropertyChanged(nameof(UnitPrice));
            OnPropertyChanged(nameof(TotalPrice));
        }
    }

    public int ZoneId
    {
        get => _zoneId;
        set { _zoneId = value; OnPropertyChanged(nameof(ZoneId)); }
    }

    public int Row
    {
        get => _row;
        set { _row = value; OnPropertyChanged(nameof(Row)); }
    }

    public int Shelf
    {
        get => _shelf;
        set { _shelf = value; OnPropertyChanged(nameof(Shelf)); }
    }
```

```
        public int Position
        {
            get => _position;
            set { _position = value; OnPropertyChanged(nameof(Position)); }
        }

        public string Notes
        {
            get => _notes;
            set { _notes = value; OnPropertyChanged(nameof(Notes)); }
        }

        public decimal TotalPrice => ReceivedQuantity * UnitPrice;

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged(string name) =>
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }

    public enum InvoiceStatus
    {
        Expected,
        PartiallyReceived,
        Received,
        Cancelled
    }
}

using System;
using System.ComponentModel;

namespace DiplomMurtazin.Model
{
    public class ReturnRequest : INotifyPropertyChanged
    {
        private int _returnId;
        private int _saleId;
        private int _productId;
        private string _productName;
        private int _quantity;
        private decimal _unitPrice;
        private decimal _refundAmount;
        private DateTime _saleDate;
        private DateTime _returnDate;
        private int _customerId;
        private string _customerName;
        private string _customerPhone;
        private ReturnReason _reason;
        private string _reasonDescription;
        private ReturnStatus _status;
        private int _processedBy;
        private string _notes;

        public int ReturnId
        {
            get => _returnId;
            set { _returnId = value; OnPropertyChanged(nameof(ReturnId)); }
        }

        public int SaleId
        {
            get => _saleId;
            set { _saleId = value; OnPropertyChanged(nameof(SaleId)); }
        }

        public int ProductId
        {
            get => _productId;
            set { _productId = value; OnPropertyChanged(nameof(ProductId)); }
        }

        public string ProductName
        {
            get => _productName;
            set { _productName = value; OnPropertyChanged(nameof(ProductName)); }
        }

        public int Quantity
        {
            get => _quantity;
            set
            {
                _quantity = value;
                OnPropertyChanged(nameof(Quantity));
                OnPropertyChanged(nameof(RefundAmount));
            }
        }

        public decimal UnitPrice
        {
            get => _unitPrice;
            set
            {
                _unitPrice = value;
                OnPropertyChanged(nameof(UnitPrice));
                OnPropertyChanged(nameof(RefundAmount));
            }
        }

        public decimal RefundAmount => Quantity * UnitPrice;

        public DateTime SaleDate
        {
            get => _saleDate;
            set { _saleDate = value; OnPropertyChanged(nameof(SaleDate)); }
        }

        public DateTime ReturnDate
        {
            get => _returnDate;
            set { _returnDate = value; OnPropertyChanged(nameof(ReturnDate)); }
        }

        public int CustomerId
        {
            get => _customerId;
            set { _customerId = value; OnPropertyChanged(nameof(CustomerId)); }
        }

        public string CustomerName
        {
            get => _customerName;
            set { _customerName = value; OnPropertyChanged(nameof(CustomerName)); }
        }

        public string CustomerPhone
        {
            get => _customerPhone;
            set { _customerPhone = value; OnPropertyChanged(nameof(CustomerPhone)); }
        }

        public ReturnReason Reason
        {
            get => _reason;
            set { _reason = value; OnPropertyChanged(nameof(Reason)); }
        }

        public string ReasonDescription
        {
            get => _reasonDescription;
            set { _reasonDescription = value; OnPropertyChanged(nameof(ReasonDescription)); }
        }

        public ReturnStatus Status
        {
```

```
            get => _status;
            set { _status = value; OnPropertyChanged(nameof(Status)); }
        }

        public int ProcessedBy
        {
            get => _processedBy;
            set { _processedBy = value;
OnPropertyChanged(nameof(ProcessedBy)); }
        }

        public string Notes
        {
            get => _notes;
            set { _notes = value; OnPropertyChanged(nameof(Notes)); }
        }

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged(string name) =>
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(name));
    }

    public enum ReturnReason
    {
        Defective,
        WrongItem,
        NotSatisfied,
        Other
    }

    public enum ReturnStatus
    {
        Pending,
        Approved,
        Rejected,
        Completed
    }
}


using DiplomMurtazin.Core;
using DiplomMurtazin.Model;
using System;
using System.Collections.ObjectModel;
using System.Data.Entity;
using System.Linq;
using System.Windows.Input;

namespace DiplomMurtazin.ViewModel
{
    public class WarehouseMapViewModel : BaseViewModel
    {
        private KPMurtazinEntities _context;
        private ObservableCollection<WarehouseMap> _maps;
        private WarehouseMap _selectedMap;
        private MapCell _selectedCell;
        private ObservableCollection<ProductLocation> _locations;
        private int _selectedZoneId;

        public ObservableCollection<WarehouseMap> Maps
        {
            get => _maps;
            set => Set(ref _maps, value);
        }

        public WarehouseMap SelectedMap
        {
            get => _selectedMap;
            set
            {
                if (Set(ref _selectedMap, value))
                {
                    LoadMapData();
                }
            }
        }

        public MapCell SelectedCell
        {
            get => _selectedCell;
            set => Set(ref _selectedCell, value);
        }

        public ObservableCollection<ProductLocation> Locations
        {
            get => _locations;
            set => Set(ref _locations, value);
        }

        public int SelectedZoneId
        {
            get => _selectedZoneId;
            set
            {
                if (Set(ref _selectedZoneId, value))
                {
                    LoadZoneMap();
                }
            }
        }

        public ICommand LoadedCommand { get; }
        public ICommand RefreshCommand { get; }
        public ICommand MoveProductCommand { get; }

        public WarehouseMapViewModel()
        {
            LoadedCommand = new RelayCommand(OnLoaded);
            RefreshCommand = new RelayCommand(Refresh);
            MoveProductCommand = new
RelayCommand(MoveProduct, CanMoveProduct);
        }

        private void OnLoaded(object parameter)
        {
            _context = new KPMurtazinEntities();
            _context.Configuration.ProxyCreationEnabled = false;
            LoadZones();
            LoadLocations();
        }

        private void LoadZones()
        {
            try
            {
                var zones = _context.WarehouseZones.ToList();
                Maps = new ObservableCollection<WarehouseMap>();

                foreach (var zone in zones)
                {
                    Maps.Add(new WarehouseMap
                    {
                        ZoneId = zone.ZoneID,
                        ZoneName = zone.ZoneName,
                        Rows = 5,
                        Shelves = 4,
                        Positions = 3,
                        Cells = new
System.Collections.Generic.List<MapCell>()
                    });
                }
            }
            catch (Exception ex)
            {
                System.Diagnostics.Debug.WriteLine($"Ошибка загрузки
зон: {ex.Message}");
            }
        }

        private void LoadLocations()
        {
            try
            {
                var query = from sb in _context.StockBalances
                            join p in _context.Products on sb.ProductID equals
p.ProductID
```

```
                join z in _context.WarehouseZones on sb.ZoneID
equals z.ZoneID
                select new ProductLocation
                {
                    ProductId = p.ProductID,
                    ProductName = p.ProductName,
                    ZoneId = z.ZoneID,
                    ZoneName = z.ZoneName,
                    Row = 1,
                    Shelf = 1,
                    Position = 1,
                    ExpectedQuantity = p.MinStockLevel ?? 5,
                    ActualQuantity = sb.Quantity
                };

            Locations = new
ObservableCollection<ProductLocation>(query.ToList());
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine($"Ошибка загрузки
расположений: {ex.Message}");
        }
    }

    private void LoadZoneMap()
    {
        if (_selectedZoneId == 0) return;

        SelectedMap = Maps.FirstOrDefault(m => m.ZoneId ==
_selectedZoneId);
        if (SelectedMap == null) return;

        var cells = new System.Collections.Generic.List<MapCell>();
        var zoneLocations = Locations.Where(l => l.ZoneId ==
_selectedZoneId).ToList();

        for (int row = 1; row <= SelectedMap.Rows; row++)
        {
            for (int shelf = 1; shelf <= SelectedMap.Shelves; shelf++)
            {
                for (int pos = 1; pos <= SelectedMap.Positions; pos++)
                {
                    var location = zoneLocations.FirstOrDefault(l =>
                        l.Row == row && l.Shelf == shelf && l.Position ==
pos);

                    var cell = new MapCell
                    {
                        Row = row,
                        Shelf = shelf,
                        Position = pos,
                        ProductId = location?.ProductId ?? 0,
                        ProductName = location?.ProductName ?? "",
                        Quantity = location?.ActualQuantity ?? 0,
                        ExpectedQuantity = location?.ExpectedQuantity ??
5
                    };

                    cells.Add(cell);
                }
            }
        }

        SelectedMap.Cells = cells;
    }

    private bool CanMoveProduct(object parameter)
    {
        return SelectedCell != null && SelectedCell.ProductId > 0;
    }

    private void MoveProduct(object parameter)
    {
        // Логика перемещения товара
    }

    private void Refresh(object parameter)
    {
        LoadZones();
        LoadLocations();
        LoadZoneMap();
    }
  }
}



using DiplomMurtazin.Core;
using DiplomMurtazin.Model;
using System;
using System.Collections.ObjectModel;
using System.Data.Entity;
using System.Linq;
using System.Windows.Input;

namespace DiplomMurtazin.ViewModel
{
  public class InvoiceViewModel : BaseViewModel
  {
    private KPMurtazinEntities _context;
    private ObservableCollection<Invoice> _invoices;
    private ObservableCollection<Invoice> _filteredInvoices;
    private Invoice _selectedInvoice;
    private ObservableCollection<Suppliers> _suppliers;
    private ObservableCollection<Products> _products;
    private Invoice _newInvoice;
    private InvoiceItem _newInvoiceItem;
    private string _searchText;
    private InvoiceStatus _selectedStatus;
    private string _statusMessage;
    private string _statusColor;

    public ObservableCollection<Invoice> Invoices
    {
        get => _invoices;
        set => Set(ref _invoices, value);
    }

    public ObservableCollection<Invoice> FilteredInvoices
    {
        get => _filteredInvoices;
        set => Set(ref _filteredInvoices, value);
    }

    public Invoice SelectedInvoice
    {
        get => _selectedInvoice;
        set
        {
            if (Set(ref _selectedInvoice, value))
            {
                LoadInvoiceDetails();
            }
        }
    }

    public ObservableCollection<Suppliers> Suppliers
    {
        get => _suppliers;
        set => Set(ref _suppliers, value);
    }

    public ObservableCollection<Products> Products
    {
        get => _products;
        set => Set(ref _products, value);
    }

    public Invoice NewInvoice
    {
        get => _newInvoice;
        set => Set(ref _newInvoice, value);
    }

    public InvoiceItem NewInvoiceItem
    {
        get => _newInvoiceItem;
```

```
        set => Set(ref _newInvoiceItem, value);
    }

    public string SearchText
    {
        get => _searchText;
        set
        {
            if (Set(ref _searchText, value))
                ApplyFilter();
        }
    }

    public InvoiceStatus SelectedStatus
    {
        get => _selectedStatus;
        set
        {
            if (Set(ref _selectedStatus, value))
                ApplyFilter();
        }
    }

    public string StatusMessage
    {
        get => _statusMessage;
        set => Set(ref _statusMessage, value);
    }

    public string StatusColor
    {
        get => _statusColor;
        set => Set(ref _statusColor, value);
    }

    public ICommand LoadedCommand { get; }
    public ICommand CreateInvoiceCommand { get; }
    public ICommand AddItemCommand { get; }
    public ICommand RemoveItemCommand { get; }
    public ICommand SaveInvoiceCommand { get; }
    public ICommand ReceiveInvoiceCommand { get; }
    public ICommand CancelInvoiceCommand { get; }
    public ICommand RefreshCommand { get; }

    public InvoiceViewModel()
    {
        LoadedCommand = new RelayCommand(OnLoaded);
        CreateInvoiceCommand = new
RelayCommand(CreateInvoice);
        AddItemCommand = new RelayCommand(AddItem,
CanAddItem);
        RemoveItemCommand = new RelayCommand(RemoveItem,
CanRemoveItem);
        SaveInvoiceCommand = new RelayCommand(SaveInvoice,
CanSaveInvoice);
        ReceiveInvoiceCommand = new
RelayCommand(ReceiveInvoice, CanReceiveInvoice);
        CancelInvoiceCommand = new
RelayCommand(CancelInvoice, CanCancelInvoice);
        RefreshCommand = new RelayCommand(Refresh);

        NewInvoice = new Invoice
        {
            InvoiceDate = DateTime.Today,
            ExpectedDate = DateTime.Today.AddDays(7),
            Status = InvoiceStatus.Expected,
            Items = new ObservableCollection<InvoiceItem>()
        };

        NewInvoiceItem = new InvoiceItem();
    }

    private void OnLoaded(object parameter)
    {
        _context = new KPMurtazinEntities();
        _context.Configuration.ProxyCreationEnabled = false;
        LoadSuppliers();
        LoadProducts();
        LoadInvoices();
    }

    private void LoadSuppliers()
    {
        try
        {
            Suppliers = new ObservableCollection<Suppliers>(
                _context.Suppliers.OrderBy(s =>
s.SupplierName).ToList()
            );
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка загрузки поставщиков:
{ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private void LoadProducts()
    {
        try
        {
            Products = new ObservableCollection<Products>(
                _context.Products.OrderBy(p =>
p.ProductName).ToList()
            );
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка загрузки товаров:
{ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private void LoadInvoices()
    {
        try
        {
            var invoices = _context.Invoices
                .Include(i => i.Suppliers)
                .OrderByDescending(i => i.InvoiceDate)
                .ToList()
                .Select(i => new Invoice
                {
                    InvoiceId = i.InvoiceID,
                    InvoiceNumber = i.InvoiceNumber,
                    SupplierId = i.SupplierID,
                    SupplierName = i.Suppliers?.SupplierName ?? "",
                    InvoiceDate = i.InvoiceDate,
                    ExpectedDate = i.ExpectedDeliveryDate,
                    ReceivedDate = i.ActualDeliveryDate,
                    Status =
(InvoiceStatus)Enum.Parse(typeof(InvoiceStatus), i.Status),
                    CreatedBy = i.CreatedBy,
                    Notes = i.Notes,
                    Items = new ObservableCollection<InvoiceItem>()
                });

            Invoices = new ObservableCollection<Invoice>(invoices);
            FilteredInvoices = new
ObservableCollection<Invoice>(invoices);
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка загрузки накладных:
{ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private void LoadInvoiceDetails()
    {
        if (SelectedInvoice == null) return;

        try
        {
            var items = _context.InvoiceItems
```

```
                .Where(ii => ii.InvoiceID == SelectedInvoice.InvoiceId)
                .Select(ii => new InvoiceItem
                {
                    InvoiceItemId = ii.InvoiceItemID,
                    InvoiceId = ii.InvoiceID,
                    ProductId = ii.ProductID,
                    ProductName = ii.Products.ProductName,
                    ExpectedQuantity = ii.ExpectedQuantity,
                    ReceivedQuantity = ii.ReceivedQuantity ?? 0,
                    UnitPrice = ii.UnitPrice,
                    Notes = ii.DiscrepancyReason
                }).ToList();

            SelectedInvoice.Items = new
ObservableCollection<InvoiceItem>(items);
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка загрузки позиций:
{ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private void CreateInvoice(object parameter)
    {
        NewInvoice = new Invoice
        {
            InvoiceDate = DateTime.Today,
            ExpectedDate = DateTime.Today.AddDays(7),
            Status = InvoiceStatus.Expected,
            Items = new ObservableCollection<InvoiceItem>()
        };
        NewInvoiceItem = new InvoiceItem();
    }

    private bool CanAddItem(object parameter)
    {
        return NewInvoiceItem.ProductId > 0 &&
NewInvoiceItem.ExpectedQuantity > 0;
    }

    private void AddItem(object parameter)
    {
        var product = Products.FirstOrDefault(p => p.ProductID ==
NewInvoiceItem.ProductId);
        if (product == null) return;

        var existingItem = NewInvoice.Items.FirstOrDefault(i =>
i.ProductId == NewInvoiceItem.ProductId);
        if (existingItem != null)
        {
            existingItem.ExpectedQuantity +=
NewInvoiceItem.ExpectedQuantity;
        }
        else
        {
            NewInvoice.Items.Add(new InvoiceItem
            {
                ProductId = product.ProductID,
                ProductName = product.ProductName,
                ExpectedQuantity = NewInvoiceItem.ExpectedQuantity,
                UnitPrice = NewInvoiceItem.UnitPrice > 0 ?
NewInvoiceItem.UnitPrice : product.UnitPrice,
                ZoneId = 1,
                Row = 1,
                Shelf = 1,
                Position = 1
            });
        }

        NewInvoiceItem = new InvoiceItem();
    }

    private bool CanRemoveItem(object parameter)
    {
        return parameter is InvoiceItem;
    }

    private void RemoveItem(object parameter)
    {
        if (parameter is InvoiceItem item)
        {
            NewInvoice.Items.Remove(item);
        }
    }

    private bool CanSaveInvoice(object parameter)
    {

return !string.IsNullOrWhiteSpace(NewInvoice.InvoiceNumber) &&
            NewInvoice.SupplierId > 0 &&
            NewInvoice.Items.Count > 0;
    }

    private void SaveInvoice(object parameter)
    {
        try
        {
            var invoice = new Invoices
            {
                SupplierID = NewInvoice.SupplierId,
                InvoiceNumber = NewInvoice.InvoiceNumber,
                InvoiceDate = NewInvoice.InvoiceDate,
                ExpectedDeliveryDate = NewInvoice.ExpectedDate,
                ShiftID = 1,
                Status = NewInvoice.Status.ToString(),
                CreatedBy = App.CurrentUser?.EmployeeID ?? 1,
                CreatedDate = DateTime.Now,
                Notes = NewInvoice.Notes
            };

            _context.Invoices.Add(invoice);
            _context.SaveChanges();

            foreach (var item in NewInvoice.Items)
            {
                var invoiceItem = new InvoiceItems
                {
                    InvoiceID = invoice.InvoiceID,
                    ProductID = item.ProductId,
                    ExpectedQuantity = item.ExpectedQuantity,
                    UnitPrice = item.UnitPrice
                };
                _context.InvoiceItems.Add(invoiceItem);
            }

            _context.SaveChanges();

            StatusMessage = $"Накладная
{NewInvoice.InvoiceNumber} создана";
            StatusColor = "#27ae60";

            LoadInvoices();
            CreateInvoice(null);
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка сохранения: {ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private bool CanReceiveInvoice(object parameter)
    {
        return SelectedInvoice != null &&
            SelectedInvoice.Status == InvoiceStatus.Expected;
    }

    private void ReceiveInvoice(object parameter)
    {
        try
        {
            var invoice =
_context.Invoices.Find(SelectedInvoice.InvoiceId);
            if (invoice == null) return;

            invoice.ActualDeliveryDate = DateTime.Now;
```

```
            invoice.Status = InvoiceStatus.Received.ToString();

            foreach (var item in SelectedInvoice.Items)
            {
                var invoiceItem = _context.InvoiceItems
                    .FirstOrDefault(ii => ii.InvoiceItemID == 
item.InvoiceItemId);

                if (invoiceItem != null)
                {
                    invoiceItem.ReceivedQuantity = 
item.ReceivedQuantity > 0 ?
                        item.ReceivedQuantity : item.ExpectedQuantity;

                    var stockBalance = _context.StockBalances
                        .FirstOrDefault(sb => sb.ProductID == 
item.ProductId &&
                                        sb.ZoneID == item.ZoneId);

                    if (stockBalance == null)
                    {
                        stockBalance = new StockBalances
                        {
                            ProductID = item.ProductId,
                            ZoneID = item.ZoneId,
                            Quantity = invoiceItem.ReceivedQuantity.Value,
                            LastUpdated = DateTime.Now
                        };
                        _context.StockBalances.Add(stockBalance);
                    }
                    else
                    {
                        stockBalance.Quantity += 
invoiceItem.ReceivedQuantity.Value;
                        stockBalance.LastUpdated = DateTime.Now;
                    }
                }
            }

            _context.SaveChanges();

            StatusMessage = $"Накладная 
{SelectedInvoice.InvoiceNumber} принята";
            StatusColor = "#27ae60";

            LoadInvoices();
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка приемки: {ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private bool CanCancelInvoice(object parameter)
    {
        return SelectedInvoice != null &&
            SelectedInvoice.Status == InvoiceStatus.Expected;
    }

    private void CancelInvoice(object parameter)
    {
        try
        {
            var invoice = 
_context.Invoices.Find(SelectedInvoice.InvoiceId);
            if (invoice == null) return;

            invoice.Status = InvoiceStatus.Cancelled.ToString();
            _context.SaveChanges();

            StatusMessage = $"Накладная 
{SelectedInvoice.InvoiceNumber} отменена";
            StatusColor = "#e74c3c";

            LoadInvoices();
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка отмены: {ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private void ApplyFilter()
    {
        if (Invoices == null) return;

        var filtered = Invoices.AsEnumerable();

        if (!string.IsNullOrWhiteSpace(SearchText))
        {
            filtered = filtered.Where(i =>
                i.InvoiceNumber.Contains(SearchText) ||
                i.SupplierName.Contains(SearchText));
        }

        filtered = filtered.Where(i => i.Status == SelectedStatus);

        FilteredInvoices = new 
ObservableCollection<Invoice>(filtered);
    }

    private void Refresh(object parameter)
    {
        _context?.Dispose();
        _context = new KPMurtazinEntities();
        _context.Configuration.ProxyCreationEnabled = false;
        LoadSuppliers();
        LoadProducts();
        LoadInvoices();
    }
  }
}
```

```
using DiplomMurtazin.Core;
using DiplomMurtazin.Model;
using System;
using System.Collections.ObjectModel;
using System.Data.Entity;
using System.Linq;
using System.Windows.Input;

namespace DiplomMurtazin.ViewModel
{
  public class ReturnViewModel : BaseViewModel
  {
    private KPMurtazinEntities _context;
    private ObservableCollection<ReturnRequest> _returns;
    private ObservableCollection<ReturnRequest> _filteredReturns;
    private ReturnRequest _selectedReturn;
    private ObservableCollection<Sales> _sales;
    private ReturnRequest _newReturn;
    private string _searchText;
    private ReturnStatus _selectedStatus;
    private string _statusMessage;
    private string _statusColor;
    private ObservableCollection<SaleItemViewModel> _saleItems;

    public ObservableCollection<ReturnRequest> Returns
    {
      get => _returns;
      set => Set(ref _returns, value);
    }

    public ObservableCollection<ReturnRequest> FilteredReturns
    {
      get => _filteredReturns;
      set => Set(ref _filteredReturns, value);
    }

    public ReturnRequest SelectedReturn
    {
      get => _selectedReturn;
      set => Set(ref _selectedReturn, value);
    }
```

```
    public ObservableCollection<Sales> Sales
    {
        get => _sales;
        set => Set(ref _sales, value);
    }

    public ReturnRequest NewReturn
    {
        get => _newReturn;
        set => Set(ref _newReturn, value);
    }

    public ObservableCollection<SaleItemViewModel> SaleItems
    {
        get => _saleItems;
        set => Set(ref _saleItems, value);
    }

    public string SearchText
    {
        get => _searchText;
        set
        {
            if (Set(ref _searchText, value))
                ApplyFilter();
        }
    }

    public ReturnStatus SelectedStatus
    {
        get => _selectedStatus;
        set
        {
            if (Set(ref _selectedStatus, value))
                ApplyFilter();
        }
    }

    public string StatusMessage
    {
        get => _statusMessage;
        set => Set(ref _statusMessage, value);
    }

    public string StatusColor
    {
        get => _statusColor;
        set => Set(ref _statusColor, value);
    }

    public ICommand LoadedCommand { get; }
    public ICommand CreateReturnCommand { get; }
    public ICommand ApproveReturnCommand { get; }
    public ICommand RejectReturnCommand { get; }
    public ICommand CompleteReturnCommand { get; }
    public ICommand LoadSaleItemsCommand { get; }
    public ICommand RefreshCommand { get; }

    public ReturnViewModel()
    {
        LoadedCommand = new RelayCommand(OnLoaded);
        CreateReturnCommand = new RelayCommand(CreateReturn,
CanCreateReturn);
        ApproveReturnCommand = new
RelayCommand(ApproveReturn, CanApproveReturn);
        RejectReturnCommand = new RelayCommand(RejectReturn,
CanRejectReturn);
        CompleteReturnCommand = new
RelayCommand(CompleteReturn, CanCompleteReturn);
        LoadSaleItemsCommand = new
RelayCommand(LoadSaleItems);
        RefreshCommand = new RelayCommand(Refresh);

        NewReturn = new ReturnRequest
        {
            ReturnDate = DateTime.Now,
            Status = ReturnStatus.Pending
        };
        SaleItems = new
ObservableCollection<SaleItemViewModel>();
    }

    private void OnLoaded(object parameter)
    {
        _context = new KPMurtazinEntities();
        _context.Configuration.ProxyCreationEnabled = false;
        LoadSales();
        LoadReturns();
    }

    private void LoadSales()
    {
        try
        {
            Sales = new ObservableCollection<Sales>(
                _context.Sales
                    .Include(s => s.Employees)
                    .OrderByDescending(s => s.SaleDateTime)
                    .Take(50)
                    .ToList()
            );
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка загрузки продаж:
{ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private void LoadReturns()
    {
        try
        {
            var returns = new ObservableCollection<ReturnRequest>();

            // В реальном приложении здесь должна быть таблица
Returns
            // Для демонстрации создаем тестовые данные
            returns.Add(new ReturnRequest
            {
                ReturnId = 1,
                SaleId = 1,
                ProductId = 1,
                ProductName = "Холодильник Samsung RB–37",
                Quantity = 1,
                UnitPrice = 45999.00m,
                SaleDate = DateTime.Now.AddDays(–5),
                ReturnDate = DateTime.Now,
                CustomerName = "Иванов Иван",
                CustomerPhone = "+7 (495) 123–45–67",
                Reason = ReturnReason.Defective,
                ReasonDescription = "Не работает компрессор",
                Status = ReturnStatus.Pending,
                Notes = "Требуется проверка"
            });

            returns.Add(new ReturnRequest
            {
                ReturnId = 2,
                SaleId = 2,
                ProductId = 5,
                ProductName = "Ноутбук ASUS ROG Zephyrus",
                Quantity = 1,
                UnitPrice = 149999.00m,
                SaleDate = DateTime.Now.AddDays(–3),
                ReturnDate = DateTime.Now,
                CustomerName = "Петров Петр",
                CustomerPhone = "+7 (495) 234–56–78",
                Reason = ReturnReason.WrongItem,
                ReasonDescription = "Не тот цвет",
                Status = ReturnStatus.Approved,
                Notes = "Одобрено"
            });

            Returns = returns;
```

```
            FilteredReturns = new ObservableCollection<ReturnRequest>(returns);
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка загрузки возвратов: {ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private void LoadSaleItems(object parameter)
    {
        if (parameter == null) return;

        int saleId = (int)parameter;
        try
        {
            var items = _context.SaleItems
                .Where(si => si.SaleID == saleId)
                .Select(si => new SaleItemViewModel
                {
                    ProductID = si.ProductID,
                    ProductName = si.Products.ProductName,
                    Quantity = si.Quantity,
                    UnitPrice = si.UnitPrice
                }).ToList();

            SaleItems = new ObservableCollection<SaleItemViewModel>(items);
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка загрузки позиций: {ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private bool CanCreateReturn(object parameter)
    {
        return NewReturn.SaleId > 0 &&
               NewReturn.ProductId > 0 &&
               NewReturn.Quantity > 0;
    }

    private void CreateReturn(object parameter)
    {
        try
        {
            NewReturn.ReturnId = Returns.Count + 1;
            NewReturn.ReturnDate = DateTime.Now;
            NewReturn.Status = ReturnStatus.Pending;

            Returns.Add(new ReturnRequest
            {
                ReturnId = NewReturn.ReturnId,
                SaleId = NewReturn.SaleId,
                ProductId = NewReturn.ProductId,
                ProductName = NewReturn.ProductName,
                Quantity = NewReturn.Quantity,
                UnitPrice = NewReturn.UnitPrice,
                SaleDate = NewReturn.SaleDate,
                ReturnDate = NewReturn.ReturnDate,
                CustomerName = NewReturn.CustomerName,
                CustomerPhone = NewReturn.CustomerPhone,
                Reason = NewReturn.Reason,
                ReasonDescription = NewReturn.ReasonDescription,
                Status = NewReturn.Status,
                Notes = NewReturn.Notes
            });

            FilteredReturns = new ObservableCollection<ReturnRequest>(Returns);

            StatusMessage = $"Заявка на возврат №{NewReturn.ReturnId} создана";
            StatusColor = "#27ae60";
            NewReturn = new ReturnRequest
            {
                ReturnDate = DateTime.Now,
                Status = ReturnStatus.Pending
            };
            SaleItems.Clear();
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка создания возврата: {ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private bool CanApproveReturn(object parameter)
    {
        return SelectedReturn != null &&
               SelectedReturn.Status == ReturnStatus.Pending;
    }

    private void ApproveReturn(object parameter)
    {
        try
        {
            SelectedReturn.Status = ReturnStatus.Approved;
            StatusMessage = $"Возврат №{SelectedReturn.ReturnId} одобрен";
            StatusColor = "#27ae60";
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка одобрения: {ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private bool CanRejectReturn(object parameter)
    {
        return SelectedReturn != null &&
               SelectedReturn.Status == ReturnStatus.Pending;
    }

    private void RejectReturn(object parameter)
    {
        try
        {
            SelectedReturn.Status = ReturnStatus.Rejected;
            StatusMessage = $"Возврат №{SelectedReturn.ReturnId} отклонен";
            StatusColor = "#e74c3c";
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка отклонения: {ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private bool CanCompleteReturn(object parameter)
    {
        return SelectedReturn != null &&
               SelectedReturn.Status == ReturnStatus.Approved;
    }

    private void CompleteReturn(object parameter)
    {
        try
        {
            SelectedReturn.Status = ReturnStatus.Completed;

            // Возврат товара на склад
            var stockBalance = _context.StockBalances
                .FirstOrDefault(sb => sb.ProductID == SelectedReturn.ProductId);

            if (stockBalance != null)
            {
                stockBalance.Quantity += SelectedReturn.Quantity;
```

```
                stockBalance.LastUpdated = DateTime.Now;
            }
            else
            {
                stockBalance = new StockBalances
                {
                    ProductID = SelectedReturn.ProductId,
                    ZoneID = 1,
                    Quantity = SelectedReturn.Quantity,
                    LastUpdated = DateTime.Now
                };
                _context.StockBalances.Add(stockBalance);
            }

            _context.SaveChanges();

            StatusMessage = $"Возврат №{SelectedReturn.ReturnId}
завершен, товар возвращен на склад";
            StatusColor = "#27ae60";
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка завершения возврата:
{ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private void ApplyFilter()
    {
        if (Returns == null) return;

        var filtered = Returns.AsEnumerable();

        if (!string.IsNullOrWhiteSpace(SearchText))
        {
            filtered = filtered.Where(r =>
                r.ProductName.Contains(SearchText) ||
                r.CustomerName.Contains(SearchText) ||
                r.CustomerPhone.Contains(SearchText));
        }

        filtered = filtered.Where(r => r.Status == SelectedStatus);

        FilteredReturns = new
ObservableCollection<ReturnRequest>(filtered);
    }

    private void Refresh(object parameter)
    {
        _context?.Dispose();
        _context = new KPMurtazinEntities();
        _context.Configuration.ProxyCreationEnabled = false;
        LoadSales();
        LoadReturns();
    }
  }
}


using DiplomMurtazin.Core;
using DiplomMurtazin.Model;
using System;
using System.Collections.ObjectModel;
using System.Data.Entity;
using System.Linq;
using System.Windows.Input;

namespace DiplomMurtazin.ViewModel
{
  public class InventoryViewModel : BaseViewModel
  {
    private KPMurtazinEntities _context;
    private ObservableCollection<InventoryAct> _inventoryActs;
    private ObservableCollection<InventoryAct> _filteredActs;
    private InventoryAct _selectedAct;
    private ObservableCollection<WarehouseZones> _zones;
    private ObservableCollection<Products> _products;
    private ObservableCollection<ProductLocation> _locations;
    private InventoryAct _newAct;
    private string _searchText;
    private string _statusMessage;
    private string _statusColor;
    private bool _isEditing;

    public ObservableCollection<InventoryAct> InventoryActs
    {
      get => _inventoryActs;
      set => Set(ref _inventoryActs, value);
    }

    public ObservableCollection<InventoryAct> FilteredActs
    {
      get => _filteredActs;
      set => Set(ref _filteredActs, value);
    }

    public InventoryAct SelectedAct
    {
      get => _selectedAct;
      set
      {
        if (Set(ref _selectedAct, value))
        {
          LoadActDetails();
        }
      }
    }

    public ObservableCollection<WarehouseZones> Zones
    {
      get => _zones;
      set => Set(ref _zones, value);
    }

    public ObservableCollection<Products> Products
    {
      get => _products;
      set => Set(ref _products, value);
    }

    public ObservableCollection<ProductLocation> Locations
    {
      get => _locations;
      set => Set(ref _locations, value);
    }

    public InventoryAct NewAct
    {
      get => _newAct;
      set => Set(ref _newAct, value);
    }

    public string SearchText
    {
      get => _searchText;
      set
      {
        if (Set(ref _searchText, value))
          ApplyFilter();
      }
    }

    public string StatusMessage
    {
      get => _statusMessage;
      set => Set(ref _statusMessage, value);
    }

    public string StatusColor
    {
      get => _statusColor;
      set => Set(ref _statusColor, value);
    }

    public bool IsEditing
    {
      get => _isEditing;
```

```
            set => Set(ref _isEditing, value);
        }

        public ICommand LoadedCommand { get; }
        public ICommand CreateInventoryCommand { get; }
        public ICommand StartInventoryCommand { get; }
        public ICommand SaveInventoryCommand { get; }
        public ICommand CompleteInventoryCommand { get; }
        public ICommand CancelInventoryCommand { get; }
        public ICommand AddLocationCommand { get; }
        public ICommand RemoveLocationCommand { get; }
        public ICommand RefreshCommand { get; }

        public InventoryViewModel()
        {
            LoadedCommand = new RelayCommand(OnLoaded);
            CreateInventoryCommand = new
RelayCommand(CreateInventory);
            StartInventoryCommand = new
RelayCommand(StartInventory, CanStartInventory);
            SaveInventoryCommand = new
RelayCommand(SaveInventory, CanSaveInventory);
            CompleteInventoryCommand = new
RelayCommand(CompleteInventory, CanCompleteInventory);
            CancelInventoryCommand = new
RelayCommand(CancelInventory, CanCancelInventory);
            AddLocationCommand = new RelayCommand(AddLocation,
CanAddLocation);
            RemoveLocationCommand = new
RelayCommand(RemoveLocation, CanRemoveLocation);
            RefreshCommand = new RelayCommand(Refresh);

            NewAct = new InventoryAct
            {
                InventoryDate = DateTime.Now,
                Status = "Подготовка",
                Locations = new
ObservableCollection<ProductLocation>()
            };
        }

        private void OnLoaded(object parameter)
        {
            _context = new KPMurtazinEntities();
            _context.Configuration.ProxyCreationEnabled = false;
            LoadZones();
            LoadProducts();
            LoadInventoryActs();
            LoadLocations();
        }

        private void LoadZones()
        {
            try
            {
                Zones = new ObservableCollection<WarehouseZones>(
                    _context.WarehouseZones.OrderBy(z =>
z.ZoneName).ToList()
                );
            }
            catch (Exception ex)
            {
                StatusMessage = $"Ошибка загрузки зон: {ex.Message}";
                StatusColor = "#e74c3c";
            }
        }

        private void LoadProducts()
        {
            try
            {
                Products = new ObservableCollection<Products>(
                    _context.Products.OrderBy(p =>
p.ProductName).ToList()
                );
            }
            catch (Exception ex)
            {
                StatusMessage = $"Ошибка загрузки товаров:
{ex.Message}";
                StatusColor = "#e74c3c";
            }
        }

        private void LoadInventoryActs()
        {
            try
            {
                var acts = new ObservableCollection<InventoryAct>();

                // Тестовые данные
                acts.Add(new InventoryAct
                {
                    InventoryId = 1,
                    InventoryDate = DateTime.Now.AddDays(–5),
                    Description = "Ежемесячная инвентаризация склада",
                    PerformedBy = 1,
                    PerformerName = "Иванов Иван",
                    ZoneId = 1,
                    ZoneName = "Зона А–1",
                    Status = "Завершена"
                });

                acts.Add(new InventoryAct
                {
                    InventoryId = 2,
                    InventoryDate = DateTime.Now,
                    Description = "Выборочная проверка",
                    PerformedBy = 2,
                    PerformerName = "Петров Петр",
                    ZoneId = 2,
                    ZoneName = "Зона А–2",
                    Status = "В процессе"
                });

                InventoryActs = acts;
                FilteredActs = new
ObservableCollection<InventoryAct>(acts);
            }
            catch (Exception ex)
            {
                StatusMessage = $"Ошибка загрузки актов:
{ex.Message}";
                StatusColor = "#e74c3c";
            }
        }

        private void LoadLocations()
        {
            try
            {
                var query = from sb in _context.StockBalances
                            join p in _context.Products on sb.ProductID equals
p.ProductID
                            join z in _context.WarehouseZones on sb.ZoneID
equals z.ZoneID
                            select new ProductLocation
                            {
                                ProductId = p.ProductID,
                                ProductName = p.ProductName,
                                ZoneId = z.ZoneID,
                                ZoneName = z.ZoneName,
                                Row = 1,
                                Shelf = 1,
                                Position = 1,
                                ExpectedQuantity = p.MinStockLevel ?? 5,
                                ActualQuantity = sb.Quantity
                            };

                Locations = new
ObservableCollection<ProductLocation>(query.ToList());
            }
            catch (Exception ex)
            {
                StatusMessage = $"Ошибка загрузки расположений:
{ex.Message}";
                StatusColor = "#e74c3c";
```

```
        }
    }

    private void LoadActDetails()
    {
        if (SelectedAct == null) return;

        IsEditing = SelectedAct.Status != "Завершена";

        // Загружаем позиции для этого акта
        var actLocations = Locations.Where(l => l.ZoneId ==
SelectedAct.ZoneId).ToList();
        SelectedAct.Locations = new
ObservableCollection<ProductLocation>(actLocations);
    }

    private void CreateInventory(object parameter)
    {
        NewAct = new InventoryAct
        {
            InventoryDate = DateTime.Now,
            Status = "Подготовка",
            Locations = new
ObservableCollection<ProductLocation>()
        };
        IsEditing = true;
    }

    private bool CanStartInventory(object parameter)
    {
        return NewAct != null &&
            NewAct.ZoneId > 0 &&
            !string.IsNullOrWhiteSpace(NewAct.Description);
    }

    private void StartInventory(object parameter)
    {
        try
        {
            NewAct.Status = "В процессе";
            NewAct.PerformedBy = App.CurrentUser?.EmployeeID ??
1;
            NewAct.PerformerName = App.CurrentUser?.Login ??
"Пользователь";

            // Загружаем текущие остатки для выбранной зоны
            var zoneLocations = Locations.Where(l => l.ZoneId ==
NewAct.ZoneId).ToList();

            foreach (var loc in zoneLocations)
            {
                NewAct.Locations.Add(new ProductLocation
                {
                    ProductId = loc.ProductId,
                    ProductName = loc.ProductName,
                    ZoneId = loc.ZoneId,
                    ZoneName = loc.ZoneName,
                    Row = loc.Row,
                    Shelf = loc.Shelf,
                    Position = loc.Position,
                    ExpectedQuantity = loc.ActualQuantity,
                    ActualQuantity = loc.ActualQuantity
                });
            }

            StatusMessage = "Инвентаризация начата";
            StatusColor = "#27ae60";
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка начала инвентаризации:
{ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private bool CanSaveInventory(object parameter)
    {
        return NewAct != null &&
            NewAct.Status == "В процессе" &&
            NewAct.Locations.Count > 0;
    }

    private void SaveInventory(object parameter)
    {
        try
        {
            StatusMessage = "Изменения сохранены";
            StatusColor = "#27ae60";
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка сохранения: {ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private bool CanCompleteInventory(object parameter)
    {
        return SelectedAct != null &&
            SelectedAct.Status == "В процессе";
    }

    private void CompleteInventory(object parameter)
    {
        try
        {
            var discrepancies = SelectedAct.Locations
                .Where(l => l.ActualQuantity != l.ExpectedQuantity)
                .ToList();

            if (discrepancies.Any())
            {
                var message = "Обнаружены расхождения:\n";
                foreach (var d in discrepancies)
                {
                    var diff = d.ActualQuantity – d.ExpectedQuantity;
                    message += $"{d.ProductName}:
{d.ExpectedQuantity} → {d.ActualQuantity} ({diff:+0;–#})\n";
                }

                // В реальном приложении здесь можно показать
диалог
            }

            // Обновляем остатки
            foreach (var loc in SelectedAct.Locations)
            {
                var stockBalance = _context.StockBalances
                    .FirstOrDefault(sb => sb.ProductID == loc.ProductId
&&
                                    sb.ZoneID == loc.ZoneId);

                if (stockBalance != null)
                {
                    stockBalance.Quantity = loc.ActualQuantity;
                    stockBalance.LastUpdated = DateTime.Now;
                }
            }

            SelectedAct.Status = "Завершена";
            IsEditing = false;

            StatusMessage = $"Инвентаризация
№{SelectedAct.InventoryId} завершена";
            StatusColor = "#27ae60";
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка завершения инвентаризации:
{ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private bool CanCancelInventory(object parameter)
    {
        return SelectedAct != null &&
```

```
            SelectedAct.Status != "Завершена";
        }

        private void CancelInventory(object parameter)
        {
            try
            {
                SelectedAct.Status = "Отменена";
                IsEditing = false;

                StatusMessage = $"Инвентаризация №{SelectedAct.InventoryId} отменена";
                StatusColor = "#e74c3c";
            }
            catch (Exception ex)
            {
                StatusMessage = $"Ошибка отмены: {ex.Message}";
                StatusColor = "#e74c3c";
            }
        }

        private bool CanAddLocation(object parameter)
        {
            return NewAct != null &&
                NewAct.Status == "Подготовка";
        }

        private void AddLocation(object parameter)
        {
            if (parameter is ProductLocation location)
            {
                NewAct.Locations.Add(new ProductLocation
                {
                    ProductId = location.ProductId,
                    ProductName = location.ProductName,
                    ZoneId = location.ZoneId,
                    ZoneName = location.ZoneName,
                    Row = location.Row,
                    Shelf = location.Shelf,
                    Position = location.Position,
                    ExpectedQuantity = location.ActualQuantity,
                    ActualQuantity = location.ActualQuantity
                });
            }
        }

        private bool CanRemoveLocation(object parameter)
        {
            return parameter is ProductLocation &&
                NewAct != null &&
                NewAct.Status == "Подготовка";
        }

        private void RemoveLocation(object parameter)
        {
            if (parameter is ProductLocation location)
            {
                NewAct.Locations.Remove(location);
            }
        }

        private void ApplyFilter()
        {
            if (InventoryActs == null) return;

            var filtered = InventoryActs.AsEnumerable();

            if (!string.IsNullOrWhiteSpace(SearchText))
            {
                filtered = filtered.Where(a =>
                    a.Description.Contains(SearchText) ||
                    a.ZoneName.Contains(SearchText) ||
                    a.PerformerName.Contains(SearchText));
            }

            FilteredActs = new ObservableCollection<InventoryAct>(filtered);
        }

        private void Refresh(object parameter)
        {
            _context?.Dispose();
            _context = new KPMurtazinEntities();
            _context.Configuration.ProxyCreationEnabled = false;
            LoadZones();
            LoadProducts();
            LoadInventoryActs();
            LoadLocations();
        }
    }
}

// Вспомогательные классы
namespace DiplomMurtazin.Model
{
    public class InventoryAct : BaseViewModel
    {
        private int _inventoryId;
        private DateTime _inventoryDate;
        private string _description;
        private int _performedBy;
        private string _performerName;
        private int _zoneId;
        private string _zoneName;
        private string _status;
        private ObservableCollection<ProductLocation> _locations;

        public int InventoryId
        {
            get => _inventoryId;
            set => Set(ref _inventoryId, value);
        }

        public DateTime InventoryDate
        {
            get => _inventoryDate;
            set => Set(ref _inventoryDate, value);
        }

        public string Description
        {
            get => _description;
            set => Set(ref _description, value);
        }

        public int PerformedBy
        {
            get => _performedBy;
            set => Set(ref _performedBy, value);
        }

        public string PerformerName
        {
            get => _performerName;
            set => Set(ref _performerName, value);
        }

        public int ZoneId
        {
            get => _zoneId;
            set => Set(ref _zoneId, value);
        }

        public string ZoneName
        {
            get => _zoneName;
            set => Set(ref _zoneName, value);
        }

        public string Status
        {
            get => _status;
            set => Set(ref _status, value);
        }

        public ObservableCollection<ProductLocation> Locations
        {
            get => _locations;
```

```
            set => Set(ref _locations, value);
        }
    }
}
```

```
using DiplomMurtazin.Core;
using DiplomMurtazin.Model;
using System;
using System.Collections.ObjectModel;
using System.Data.Entity;
using System.Linq;
using System.Windows.Input;

namespace DiplomMurtazin.ViewModel
{
    public class DashboardViewModel : BaseViewModel
    {
        private KPMurtazinEntities _context;
        private decimal _totalSalesToday;
        private int _totalTransactionsToday;
        private int _productsInStock;
        private int _lowStockProducts;
        private int _pendingReturns;
        private int _pendingInvoices;
        private  ObservableCollection<SaleItemViewModel>
_topProducts;
        private ObservableCollection<Sales> _recentSales;
        private            ObservableCollection<InventoryAct>
_recentInventory;
        private DateTime _selectedDate;
        private string _statusMessage;
        private string _statusColor;

        public decimal TotalSalesToday
        {
            get => _totalSalesToday;
            set => Set(ref _totalSalesToday, value);
        }

        public int TotalTransactionsToday
        {
            get => _totalTransactionsToday;
            set => Set(ref _totalTransactionsToday, value);
        }

        public int ProductsInStock
        {
            get => _productsInStock;
            set => Set(ref _productsInStock, value);
        }

        public int LowStockProducts
        {
            get => _lowStockProducts;
            set => Set(ref _lowStockProducts, value);
        }

        public int PendingReturns
        {
            get => _pendingReturns;
            set => Set(ref _pendingReturns, value);
        }

        public int PendingInvoices
        {
            get => _pendingInvoices;
            set => Set(ref _pendingInvoices, value);
        }

        public   ObservableCollection<SaleItemViewModel>
TopProducts
        {
            get => _topProducts;
            set => Set(ref _topProducts, value);
        }

        public ObservableCollection<Sales> RecentSales
        {
            get => _recentSales;
            set => Set(ref _recentSales, value);
        }

        public            ObservableCollection<InventoryAct>
RecentInventory
        {
            get => _recentInventory;
            set => Set(ref _recentInventory, value);
        }

        public DateTime SelectedDate
        {
            get => _selectedDate;
            set
            {
                if (Set(ref _selectedDate, value))
                {
                    LoadDashboardData();
                }
            }
        }

        public string StatusMessage
        {
            get => _statusMessage;
            set => Set(ref _statusMessage, value);
        }

        public string StatusColor
        {
            get => _statusColor;
            set => Set(ref _statusColor, value);
        }

        public ICommand LoadedCommand { get; }
        public ICommand RefreshCommand { get; }
        public ICommand ExportReportCommand { get; }

        public DashboardViewModel()
        {
            LoadedCommand                =                new
RelayCommand(OnLoaded);
            RefreshCommand               =                new
RelayCommand(Refresh);
            ExportReportCommand              =              new
RelayCommand(ExportReport);

            SelectedDate = DateTime.Today;
        }

        private void OnLoaded(object parameter)
        {
            _context = new KPMurtazinEntities();
            _context.Configuration.ProxyCreationEnabled    =
false;
            LoadDashboardData();
        }

        private void LoadDashboardData()
        {
            try
            {
                // Продажи за сегодня
                var todaySales = _context.Sales
                    .Where(s                                =>
DbFunctions.TruncateTime(s.SaleDateTime) == SelectedDate.Date)
                    .ToList();

                TotalSalesToday    =    todaySales.Sum(s    =>
s.TotalAmount);
                TotalTransactionsToday = todaySales.Count;

                // Товары на складе
                ProductsInStock                             =
_context.StockBalances.Sum(sb => sb.Quantity);

                // Товары с низким остатком
```

```
                LowStockProducts = (from p in _context.Products
                        join sb in _context.StockBalances on p.ProductID equals sb.ProductID into stocks
                        from sb in stocks.DefaultIfEmpty()
                        where (sb == null ? 0 : sb.Quantity) < p.MinStockLevel
                        select p).Count();

                // Ожидающие возвраты (в реальном приложении – из таблицы Returns)
                PendingReturns = 3; // Заглушка

                // Ожидающие накладные
                PendingInvoices = _context.Invoices
                    .Count(i => i.Status == "Ожидает приёмки");

                // Топ–5 товаров
                var topProductsData = _context.SaleItems
                    .GroupBy(si => si.ProductID)
                    .Select(g => new
                    {
                        ProductId = g.Key,
                        ProductName = g.First().Products.ProductName,
                        TotalSold = g.Sum(si => si.Quantity),
                        TotalRevenue = g.Sum(si => si.TotalPrice)
                    })
                    .OrderByDescending(x => x.TotalSold)
                    .Take(5)
                    .ToList();

                TopProducts = new ObservableCollection<SaleItemViewModel>();
                foreach (var item in topProductsData)
                {
                    TopProducts.Add(new SaleItemViewModel
                    {
                        ProductID = item.ProductId,
                        ProductName = item.ProductName,
                        Quantity = item.TotalSold,
                        UnitPrice = item.TotalRevenue / item.TotalSold
                    });
                }

                // Последние продажи
                RecentSales = new ObservableCollection<Sales>(
                    _context.Sales
                        .Include(s => s.Employees)
                        .OrderByDescending(s => s.SaleDateTime)
                        .Take(10)
                        .ToList()
                );

                // Последние инвентаризации (заглушка)
                RecentInventory = new ObservableCollection<InventoryAct>
                {
                    new InventoryAct
                    {
                        InventoryId = 1,
                        InventoryDate = DateTime.Now.AddDays(–2),
                        ZoneName = "Зона А–1",
                        Status = "Завершена"
                    },
                    new InventoryAct
                    {
                        InventoryId = 2,
                        InventoryDate = DateTime.Now.AddDays(–1),
                        ZoneName = "Зона Б–2",
                        Status = "В процессе"
                    }
                };

                StatusMessage = "Данные обновлены";
                StatusColor = "#27ae60";
            }
            catch (Exception ex)
            {
                StatusMessage = $"Ошибка загрузки данных: {ex.Message}";
                StatusColor = "#e74c3c";
            }
        }

        private void Refresh(object parameter)
        {
            _context?.Dispose();
            _context = new KPMurtazinEntities();
            _context.Configuration.ProxyCreationEnabled = false;
            LoadDashboardData();
        }

        private void ExportReport(object parameter)
        {
            try
            {
                var dialog = new Microsoft.Win32.SaveFileDialog
                {
                    Filter = "CSV files (*.csv)|*.csv",
                    FileName = $"Отчет_{SelectedDate:yyyyMMdd}.csv"
                };

                if (dialog.ShowDialog() == true)
                {
                    var sb = new System.Text.StringBuilder();
                    sb.AppendLine("Показатель,Значение");
                    sb.AppendLine($"Продажи за день,{TotalSalesToday:F2}");
                    sb.AppendLine($"Количество продаж,{TotalTransactionsToday}");
                    sb.AppendLine($"Товаров на складе,{ProductsInStock}");
                    sb.AppendLine($"Товаров ниже минимума,{LowStockProducts}");
                    sb.AppendLine($"Ожидает возвратов,{PendingReturns}");
                    sb.AppendLine($"Ожидает накладных,{PendingInvoices}");

                    sb.AppendLine();
                    sb.AppendLine("Топ товаров:");
                    sb.AppendLine("Наименование,Количество,Выручка");

                    foreach (var item in TopProducts)
                    {
                        sb.AppendLine($"{item.ProductName},{item.Quantity},{item.TotalPrice:F2}");
                    }

                    System.IO.File.WriteAllText(dialog.FileName, sb.ToString(), System.Text.Encoding.UTF8);

                    StatusMessage = $"Отчет сохранен: {dialog.FileName}";
                    StatusColor = "#27ae60";
                }
            }
            catch (Exception ex)
            {
                StatusMessage = $"Ошибка экспорта: {ex.Message}";
                StatusColor = "#e74c3c";
            }
        }
    }
}
```

```
using DiplomMurtazin.Core;
using DiplomMurtazin.Model;
using System;
using System.Collections.ObjectModel;
using System.Data.Entity;
using System.Linq;
using System.Windows.Input;

namespace DiplomMurtazin.ViewModel
{
  public class ProductMovementViewModel :
BaseViewModel
  {
    private KPMurtazinEntities _context;
    private
ObservableCollection<ProductMovementHistory> _movements;
    private
ObservableCollection<ProductMovementHistory>
_filteredMovements;
    private ObservableCollection<Products> _products;
    private Products _selectedProduct;
    private DateTime _startDate;
    private DateTime _endDate;
    private string _movementType;
    private string _searchText;
    private string _statusMessage;
    private string _statusColor;

    public
ObservableCollection<ProductMovementHistory> Movements
    {
      get => _movements;
      set => Set(ref _movements, value);
    }

    public
ObservableCollection<ProductMovementHistory>
FilteredMovements
    {
      get => _filteredMovements;
      set => Set(ref _filteredMovements, value);
    }

    public ObservableCollection<Products> Products
    {
      get => _products;
      set => Set(ref _products, value);
    }

    public Products SelectedProduct
    {
      get => _selectedProduct;
      set
      {
        if (Set(ref _selectedProduct, value))
        {
          LoadMovements();
        }
      }
    }

    public DateTime StartDate
    {
      get => _startDate;
      set
      {
        if (Set(ref _startDate, value))
        {
          LoadMovements();
        }
      }
    }

    public DateTime EndDate
    {
      get => _endDate;
      set
      {
        if (Set(ref _endDate, value))
        {
          LoadMovements();
        }
      }
    }

    public string MovementType
    {
      get => _movementType;
      set
      {
        if (Set(ref _movementType, value))
        {
          LoadMovements();
        }
      }
    }

    public string SearchText
    {
      get => _searchText;
      set
      {
        if (Set(ref _searchText, value))
          ApplyFilter();
      }
    }

    public string StatusMessage
    {
      get => _statusMessage;
      set => Set(ref _statusMessage, value);
    }

    public string StatusColor
    {
      get => _statusColor;
      set => Set(ref _statusColor, value);
    }

    public ICommand LoadedCommand { get; }
    public ICommand RefreshCommand { get; }
    public ICommand ExportCommand { get; }
    public ICommand ClearFiltersCommand { get; }

    public ProductMovementViewModel()
    {
      LoadedCommand = new
RelayCommand(OnLoaded);
      RefreshCommand = new
RelayCommand(Refresh);
      ExportCommand = new RelayCommand(Export);
      ClearFiltersCommand = new
RelayCommand(ClearFilters);

      StartDate = DateTime.Today.AddMonths(–1);
      EndDate = DateTime.Today;
      MovementType = "Все";
    }

    private void OnLoaded(object parameter)
    {
      _context = new KPMurtazinEntities();
      _context.Configuration.ProxyCreationEnabled =
false;
      LoadProducts();
      LoadMovements();
    }

    private void LoadProducts()
    {
      try
      {
        Products = new
ObservableCollection<Products>(
          _context.Products.OrderBy(p =>
p.ProductName).ToList()
        );
```

```
                Products.Insert(0, new Products { ProductID = 0,
ProductName = "Все товары" });
            }
            catch (Exception ex)
            {
                StatusMessage = $"Ошибка загрузки товаров:
{ex.Message}";
                StatusColor = "#e74c3c";
            }
        }

        private void LoadMovements()
        {
            try
            {
                var query = _context.ProductMovementHistory
                    .Include(m => m.Products)
                    .Include(m => m.Employees)
                    .Where(m => m.MovementDate >= StartDate
&& m.MovementDate <= EndDate.AddDays(1));

                if (SelectedProduct != null &&
SelectedProduct.ProductID > 0)
                {
                    query = query.Where(m => m.ProductID ==
SelectedProduct.ProductID);
                }

                if (!string.IsNullOrWhiteSpace(MovementType)
&& MovementType != "Все")
                {
                    query = query.Where(m => m.MovementType
== MovementType);
                }

                Movements = new
ObservableCollection<ProductMovementHistory>(
                    query.OrderByDescending(m =>
m.MovementDate).ToList()
                );

                FilteredMovements = new
ObservableCollection<ProductMovementHistory>(Movements);

                StatusMessage = $"Загружено записей:
{Movements.Count}";
                StatusColor = "#3498db";
            }
            catch (Exception ex)
            {
                StatusMessage = $"Ошибка загрузки
движений: {ex.Message}";
                StatusColor = "#e74c3c";
            }
        }

        private void ApplyFilter()
        {
            if (Movements == null) return;

            var filtered = Movements.AsEnumerable();

            if (!string.IsNullOrWhiteSpace(SearchText))
            {
                filtered = filtered.Where(m =>
                    (m.Products != null &&
m.Products.ProductName.Contains(SearchText)) ||
                    (m.Employees != null &&
(m.Employees.LastName.Contains(SearchText) ||

m.Employees.FirstName.Contains(SearchText))) ||

m.SourceDocumentNumber.Contains(SearchText));
            }

            FilteredMovements = new
ObservableCollection<ProductMovementHistory>(filtered);
        }

        private void ClearFilters(object parameter)
        {
            SelectedProduct = Products?.FirstOrDefault(p =>
p.ProductID == 0);
            StartDate = DateTime.Today.AddMonths(–1);
            EndDate = DateTime.Today;
            MovementType = "Все";
            SearchText = "";

            LoadMovements();
        }

        private void Refresh(object parameter)
        {
            _context?.Dispose();
            _context = new KPMurtazinEntities();
            _context.Configuration.ProxyCreationEnabled =
false;
            LoadProducts();
            LoadMovements();
        }

        private void Export(object parameter)
        {
            try
            {
                var dialog = new
Microsoft.Win32.SaveFileDialog
                {
                    Filter = "CSV files (*.csv)|*.csv",
                    FileName =
$"Движение_товаров_{DateTime.Now:yyyyMMddHHmmss}.csv"
                };

                if (dialog.ShowDialog() == true)
                {
                    var sb = new System.Text.StringBuilder();
                    sb.AppendLine("Дата;Товар;Тип
движения;Количество;Документ;Сотрудник");

                    foreach (var m in FilteredMovements)
                    {

sb.AppendLine($"{m.MovementDate:dd.MM.yyyy HH:mm};" +
                        $"{m.Products?.ProductName};" +
                        $"{m.MovementType};" +
                        $"{m.Quantity};" +
                        $"{m.SourceDocumentNumber};"
+
                        $"{m.Employees?.LastName}
{m.Employees?.FirstName}");
                    }


System.IO.File.WriteAllText(dialog.FileName, sb.ToString(),
System.Text.Encoding.UTF8);

                    StatusMessage = $"Данные
экспортированы: {dialog.FileName}";
                    StatusColor = "#27ae60";
                }
            }
            catch (Exception ex)
            {
                StatusMessage = $"Ошибка экспорта:
{ex.Message}";
                StatusColor = "#e74c3c";
            }
        }
    }
}


using DiplomMurtazin.Core;
using DiplomMurtazin.Model;
using System;
using System.Collections.ObjectModel;
using System.Data.Entity;
```

```
using System.Linq;
using System.Windows.Input;

namespace DiplomMurtazin.ViewModel
{
    public class NotificationViewModel : BaseViewModel
    {
        private KPMurtazinEntities _context;
        private ObservableCollection<Notification> _notifications;
        private ObservableCollection<Notification> _unreadNotifications;
        private Notification _selectedNotification;
        private int _unreadCount;
        private string _statusMessage;
        private string _statusColor;

        public ObservableCollection<Notification> Notifications
        {
            get => _notifications;
            set => Set(ref _notifications, value);
        }

        public ObservableCollection<Notification> UnreadNotifications
        {
            get => _unreadNotifications;
            set => Set(ref _unreadNotifications, value);
        }

        public Notification SelectedNotification
        {
            get => _selectedNotification;
            set => Set(ref _selectedNotification, value);
        }

        public int UnreadCount
        {
            get => _unreadCount;
            set => Set(ref _unreadCount, value);
        }

        public string StatusMessage
        {
            get => _statusMessage;
            set => Set(ref _statusMessage, value);
        }

        public string StatusColor
        {
            get => _statusColor;
            set => Set(ref _statusColor, value);
        }

        public ICommand LoadedCommand { get; }
        public ICommand MarkAsReadCommand { get; }
        public ICommand MarkAllAsReadCommand { get; }
        public ICommand DeleteNotificationCommand { get; }
        public ICommand RefreshCommand { get; }

        public NotificationViewModel()
        {
            LoadedCommand = new RelayCommand(OnLoaded);
            MarkAsReadCommand = new RelayCommand(MarkAsRead, CanMarkAsRead);
            MarkAllAsReadCommand = new RelayCommand(MarkAllAsRead);
            DeleteNotificationCommand = new RelayCommand(DeleteNotification, CanDeleteNotification);
            RefreshCommand = new RelayCommand(Refresh);
        }

        private void OnLoaded(object parameter)
        {
            _context = new KPMurtazinEntities();
            _context.Configuration.ProxyCreationEnabled = false;
            LoadNotifications();
        }

        private void LoadNotifications()
        {
            try
            {
                var notifications = new ObservableCollection<Notification>();

                // Товары с низким остатком
                var lowStockProducts = (from p in _context.Products
                                        join sb in _context.StockBalances on p.ProductID equals sb.ProductID into stocks
                                        from sb in stocks.DefaultIfEmpty()
                                        where (sb == null ? 0 : sb.Quantity) < p.MinStockLevel
                                        select new
                                        {
                                            Product = p,
                                            Stock = sb == null ? 0 : sb.Quantity
                                        }).ToList();

                foreach (var item in lowStockProducts)
                {
                    notifications.Add(new Notification
                    {
                        Id = notifications.Count + 1,
                        Type = "Низкий остаток",
                        Title = $"Низкий остаток: {item.Product.ProductName}",
                        Message = $"Текущий остаток: {item.Stock}, минимальный уровень: {item.Product.MinStockLevel}",
                        CreatedDate = DateTime.Now,
                        IsRead = false,
                        Priority = NotificationPriority.High,
                        RelatedId = item.Product.ProductID,
                        RelatedType = "Product"
                    });
                }

                // Ожидающие накладные
                var pendingInvoices = _context.Invoices
                    .Where(i => i.Status == "Ожидает приёмки")
                    .Take(5)
                    .ToList();

                foreach (var invoice in pendingInvoices)
                {
                    notifications.Add(new Notification
                    {
                        Id = notifications.Count + 1,
                        Type = "Накладная",
                        Title = $"Накладная {invoice.InvoiceNumber} ожидает приемки",
                        Message = $"Поставщик: {invoice.Suppliers?.SupplierName}, дата: {invoice.ExpectedDeliveryDate:dd.MM.yyyy}",
                        CreatedDate = DateTime.Now,
                        IsRead = false,
                        Priority = NotificationPriority.Medium,
                        RelatedId = invoice.InvoiceID,
                        RelatedType = "Invoice"
                    });
                }

                // Возвраты (заглушка)
                notifications.Add(new Notification
                {
                    Id = notifications.Count + 1,
                    Type = "Возврат",
                    Title = "Новая заявка на возврат",
                    Message = "Клиент Иванов И.И. хочет вернуть товар",
```

```
                CreatedDate = DateTime.Now.AddHours(–2),
                IsRead = false,
                Priority = NotificationPriority.Medium
            });

            notifications.Add(new Notification
            {
                Id = notifications.Count + 1,
                Type = "Система",
                Title = "Обновление данных",
                Message = "Рекомендуется провести
инвентаризацию",
                CreatedDate = DateTime.Now.AddDays(–1),
                IsRead = true,
                Priority = NotificationPriority.Low
            });

            Notifications = new
ObservableCollection<Notification>(
                notifications.OrderByDescending(n =>
n.CreatedDate)
            );

            UpdateUnreadCount();
        }
        catch (Exception ex)
        {
            StatusMessage = $"Ошибка загрузки
уведомлений: {ex.Message}";
            StatusColor = "#e74c3c";
        }
    }

    private void UpdateUnreadCount()
    {
        UnreadCount = Notifications?.Count(n
=> !n.IsRead) ?? 0;
        UnreadNotifications = new
ObservableCollection<Notification>(
            Notifications?.Where(n
=> !n.IsRead).ToList() ?? new List<Notification>()
        );
    }

    private bool CanMarkAsRead(object parameter)
    {
        return SelectedNotification != null
&& !SelectedNotification.IsRead;
    }

    private void MarkAsRead(object parameter)
    {
        if (SelectedNotification == null) return;

        SelectedNotification.IsRead = true;
        UpdateUnreadCount();

        StatusMessage = "Уведомление отмечено как
прочитанное";
        StatusColor = "#27ae60";
    }

    private void MarkAllAsRead(object parameter)
    {
        foreach (var notification in Notifications)
        {
            notification.IsRead = true;
        }
        UpdateUnreadCount();

        StatusMessage = "Все уведомления отмечены
как прочитанные";
        StatusColor = "#27ae60";
    }

    private bool CanDeleteNotification(object parameter)
    {
        return SelectedNotification != null;
    }

    private void DeleteNotification(object parameter)
    {
        if (SelectedNotification == null) return;

        Notifications.Remove(SelectedNotification);
        UpdateUnreadCount();

        StatusMessage = "Уведомление удалено";
        StatusColor = "#27ae60";
    }

    private void Refresh(object parameter)
    {
        LoadNotifications();
    }
}

public class Notification : BaseViewModel
{
    private int _id;
    private string _type;
    private string _title;
    private string _message;
    private DateTime _createdDate;
    private bool _isRead;
    private NotificationPriority _priority;
    private int _relatedId;
    private string _relatedType;

    public int Id
    {
        get => _id;
        set => Set(ref _id, value);
    }

    public string Type
    {
        get => _type;
        set => Set(ref _type, value);
    }

    public string Title
    {
        get => _title;
        set => Set(ref _title, value);
    }

    public string Message
    {
        get => _message;
        set => Set(ref _message, value);
    }

    public DateTime CreatedDate
    {
        get => _createdDate;
        set => Set(ref _createdDate, value);
    }

    public bool IsRead
    {
        get => _isRead;
        set => Set(ref _isRead, value);
    }

    public NotificationPriority Priority
    {
        get => _priority;
        set => Set(ref _priority, value);
    }

    public int RelatedId
    {
        get => _relatedId;
        set => Set(ref _relatedId, value);
    }

    public string RelatedType
```

```
        {
            get => _relatedType;
            set => Set(ref _relatedType, value);
        }

        public string PriorityColor
        {
            get
            {
                switch (Priority)
                {
                    case NotificationPriority.High:
                        return "#e74c3c";
                    case NotificationPriority.Medium:
                        return "#f39c12";
                    case NotificationPriority.Low:
                        return "#3498db";
                    default:
                        return "#7f8c8d";
                }
            }
        }
    }

    public enum NotificationPriority
    {
        Low,
        Medium,
        High
    }
}
```